BULACAN STATE UNIVERSITY

College of Information and Communications Technology

Test Cases and Traceability

Campus Equipment Borrowing and Inventory System

Official PDF-derived scenarios, executed evidence and explicit verification gaps

Submission detail	Value
Course / academic year	IT 401W - Web Programming and Frameworks | 2026-2027
Group	Group 1
Section and proponents	To be completed with the verified section, roster and student identifiers before submission.
Prepared snapshot	25 October 2026 | Python 3.13.7 | Django 6.1.2
Submitted to	Dr. Aaron Paul M. Dela Rosa, as named in the official project specification.

Evidence-qualified edition. This document describes the copied implementation and tests executed for run IT401W-G1-20261025. Unexecuted evaluations and unavailable academic records are explicitly identified.

Contents

1. Test basis and execution record 4

1.1 Authoritative specification and test design 4

1.2 Status and coverage interpretation 5

1.3 Execution and reproduction 5

1.4 Q evidence: observed build and artifact checks 6

2. Requirements traceability matrix 7

2.1 Requirements without any verified passing scenario 12

2.2 Requirements with mixed verification 12

3. Requirement-derived scenario records 13

A1 - Accounts 13

A2 - Catalog and physical units 16

A3 - Request review 23

A4 - Equipment release 26

A5 - Returns and overdue 29

A6 - Maintenance and reconciliation 32

A7 - Dashboard and reports 36

A8 - Audit and settings 39

U1 - Authentication and profile 42

U2 - Equipment browsing 45

U3 - Request submission 48

U4 - Tracking and cancellation 52

BR1 - Atomic reservation 54

BR2 - Physical-unit status 55

BR3 - Concurrent approvals 56

BR4 - All-row validation . . . 57

BR5 - Quantity/duration and revision . . . 58

BR6 - Calendar-day overdue . . . 60

RA - Shared access control . . . 62

UI - Interface usability . . . 63

DB - Database representation/integrity . . . 65

TECH - Framework and security . . . 68

DEMO - Required fictional demonstration . . . 71

SUB - Submission delivery . . . 72

EVID - Academic and professional evidence . . . 73

4. Exact automated evidence register 76

4.1 Complete suite scope beyond selected references . . . 93

5. Summary, defects and outstanding verification 94

5.1 Before the instructor demonstration . . . 95

5.2 Source reference . . . 95

1. Test basis and execution record

1.1 Authoritative specification and test design

This register is derived from the five-page IT 401W Equipment Borrowing specification, first semester AY 2026-2027, Bulacan State University. Original identifiers A1-A8, U1-U4 and BR1-BR6 are preserved. Sub-identifiers such as A2.4 and the unnumbered-prose locators RA, UI, DB, TECH, DEMO, SUB and EVID are documentation locators, not new instructor requirements. Each of the 116 scenarios links to at least one of 80 normalized mandatory checks. [R1]

Expected results come from the specification. Actual source/schema determines the implementation mapping; recorded execution determines the result. The logical Profile is embedded in accounts.User and per-unit ReturnRecord in loans.LoanUnit. Allocation has no created_at field: approval timing derives from the request and end tracking from cancellation/return relations. Exact schema and diagrams are in the companion Documentation PDF.

Run attribute	Observed evidence
Run / date	IT401W-G1-20261025 | 25 October 2026 (Asia/Manila)
Environment	Python 3.13.7; Django 6.1.2; SQLite; Asia/Manila; separate pinned-dependency environment.
Automated run	524 methods executed; 524 PASS; 0 FAIL; 0 skipped. Runner time: 63.894 seconds.
Isolation	Test databases and fictional screenshot database were outside the working application database. No original application code was changed to make assertions pass.
Checks	Django check: no issues; migration consistency: no changes detected; pip check: no broken requirements; new isolated migration/seed verification succeeded.
Actual browser evidence	26 PNGs, all 18 required names plus 8 supplementary views, at a configured 1440 x 1100 desktop viewport. Browser content capture bounds are 1440 x 1061 for 23 images and 1425 x 1050 for 3 images. These show real pages; screenshots alone do not prove unperformed mutations.
Source PDF	Five pages; SHA-256 77903c0959180782d2b94639d24e824155d0edf08ef65caa76222573a4502bff

1.2 Status and coverage interpretation

Status	Meaning
PASS	The exact cited automated assertion(s) or artifact check actually passed in this preparation run.
FAIL	An executed required assertion/check failed. No such result was observed in this run.
NOT EXECUTED	The scenario has a procedure but its actual outcome was not exercised; missing evidence is not treated as success.
BLOCKED	Required external information, participants, signatures, policy/channel or academic records were not available.
PARTIAL (matrix only)	The requirement has at least one verified passing scenario and another scenario without a passing result. It is not full verified coverage.

Counting rule
The 524 test methods are not 524 PDF requirements and are not the same as the 116 scenario records. Scenarios group related positive, negative and boundary assertions. A method may support several requirements. Automated cases describe reproducible service/HTTP procedures; they do not claim a human clicked through every listed step.

PASS on an automated case means the listed assertion methods passed; inspect the permanent test source for exact fixture values and assertions. Each case below includes its own preconditions/data/steps/expected/actual/status/evidence. The T-number register resolves evidence to exact included test file, line and fully qualified method. Build checks are recorded separately as Q evidence. Temporary runner logs and private configuration are deliberately excluded from the final source ZIP.

1.3 Execution and reproduction

1. Extract the authoritative ZIP. Follow README to install the pinned requirements in a Python 3.13 environment and provision the isolated fictional demo with scripts/demo.py setup.

2. Run python scripts/demo.py check, python scripts/demo.py makemigrations --check --dry-run and python -m pip check using that environment.

3. Run python scripts/demo.py test --noinput to execute the complete permanent test suite. For a subset, pass an exact fully qualified method or module from the evidence register.

4. Use only isolated fixtures for destructive/conflicting test scenarios. Never point concurrency or seed tests at operational/development data.

5. Record the new date/environment and actual output. A later run is independent evidence; it must not silently inherit this run's PASS labels.

1.4 Q evidence: observed build and artifact checks

Evidence key	Check and scope
Q-schema	Actual twelve-model metadata and migrated isolated schema; includes model fields/relations/constraints, logical Profile/Return mapping and sixteen domain migration files. VERIFIED.
Q-framework	Inspected config/URLs/models/forms/services/templates. Django template backend and inherited role bases exist; /admin/ does not resolve. VERIFIED.
Q-dependencies	New environment installed pinned requirements, pip check passed; copied app migrated/seeded and served on loopback independently of the working DB. VERIFIED.
Q-no_finance	Actual domain registry, overdue code and routes contain no payment/fine/procurement/supplier subsystem; overdue is status/day calculation only. VERIFIED.
Q-screenshots	All 26 actual PNGs exist and were visually reviewed as desktop evidence. Configured viewport: 1440 x 1100; actual browser content bounds: 1440 x 1061 (23) or 1425 x 1050 (3). No password-bearing screenshot is included. VERIFIED.
Q-pdfs	Final PDFs reopened/extracted/rendered; page layout, TOC, page numbers, diagrams/tables/screenshots reviewed. Status reflects the final artifact validation flag. VERIFIED.
Q-package	ZIP opens and tests successfully; required paths/member hashes/exclusions and source preservation checked. Status reflects final package validation flag. VERIFIED.

2. Requirements traceability matrix

Every normalized mandatory check has one or more linked scenarios. Source page references are included in the requirement description. All scenarios and their current outcomes contribute to the matrix status; a single passing case does not hide an unexecuted required evaluation. Sub-identifiers are author-created locators under the official parent or prose section.

Requirement	Requirement and source page	Scenario IDs	Result
A1.1	List and search Basic User accounts. Official IT 401W PDF, p.1, A1	TC-A1-001	PASS
A1.2	Activate/deactivate without deleting records. Official IT 401W PDF, p.1, A1	TC-A1-002	PASS
A1.3	Show account request history. Official IT 401W PDF, p.1, A1	TC-A1-003	PASS
A1.4	Inactive accounts cannot sign in or transact, including existing sessions. Official IT 401W PDF, p.1, A1	TC-A1-004, TC-A1-005	PASS
A1.5	Registration cannot offer or assign Admin. Official IT 401W PDF, p.1, A1	TC-A1-006	PASS
A2.1	Create/view/update/archive categories. Official IT 401W PDF, p.2, A2	TC-A2-001, TC-A2-002, TC-A2-003, TC-A2-004	PASS
A2.2	Create/view/update/archive catalog items. Official IT 401W PDF, p.2, A2	TC-A2-005, TC-A2-006, TC-A2-007	PASS
A2.3	Register physical units with unique asset codes. Official IT 401W PDF, p.2, A2	TC-A2-008	PASS
A2.4	Store condition, acquisition date and inventory status. Official IT 401W PDF, p.2, A2	TC-A2-009, TC-A2-010	PASS
A2.5	Filter physical units by category, item and status. Official IT 401W PDF, p.2, A2	TC-A2-011, TC-A2-012	PASS
A2.6	Preserve referenced masters and transaction relationships. Official IT 401W PDF, p.2, A2	TC-A2-006, TC-A2-013	PASS
A3.1	Searchable paginated queue with status/date filters. Official IT 401W PDF, p.2, A3	TC-A3-001, TC-A3-002	PASS
A3.2	Review quantities, desired date, purpose and current availability. Official IT 401W PDF, p.2, A3	TC-A3-003	PASS
A3.3	Approval/rejection records a decision and reason. Official IT 401W PDF, p.2, A3	TC-A3-004, TC-A3-005	PASS
A3.4	Approval allocates specific available units and reserves them. Official IT 401W PDF, p.2, A3	TC-A3-006	PASS

Requirement	Requirement and source page	Scenario IDs	Result
A4.1	Convert Approved request into one loan with actor/date/units/due date. Official IT 401W PDF, p.2, A4	TC-A4-001	PASS
A4.2	Printable slip contains borrower and asset codes. Official IT 401W PDF, p.2, A4	TC-A4-002, TC-A4-003	PARTIAL
A4.3	Reject invalid, duplicate and insufficiently allocated release. Official IT 401W PDF, p.2, A4	TC-A4-004, TC-A4-005	PASS
A5.1	Per-unit returns record date and condition. Official IT 401W PDF, p.2, A5	TC-A5-001, TC-A5-003	PASS
A5.2	Partial returns retain outstanding units; close after the last return. Official IT 401W PDF, p.2, A5	TC-A5-001, TC-A5-002	PASS
A5.3	Outstanding loans are overdue after the local due calendar date. Official IT 401W PDF, p.2, A5	TC-A5-004	PASS
A5.4	Damaged returns enter Maintenance. Official IT 401W PDF, p.2, A5	TC-A5-005	PASS
A6.1	Maintenance records retain problem/action/dates/Admin. Official IT 401W PDF, p.2, A6	TC-A6-001, TC-A6-002, TC-A6-005	PASS
A6.2	Repaired units become Available; retired units remain unavailable. Official IT 401W PDF, p.2, A6	TC-A6-003, TC-A6-004, TC-A6-005	PASS
A6.3	Reject maintenance for Reserved or Issued units. Official IT 401W PDF, p.2, A6	TC-A6-006	PASS
A6.4	Derived five-status counts reconcile to physical units. Official IT 401W PDF, p.2, A6	TC-A6-007	PASS
A7.1	Four live KPIs: available units, pending requests, active loans, overdue loans. Official IT 401W PDF, p.2, A7	TC-A7-001	PASS
A7.2	Monthly issued-loan chart. Official IT 401W PDF, p.2, A7	TC-A7-002	PASS
A7.3	Equipment-category usage chart. Official IT 401W PDF, p.2, A7	TC-A7-003	PASS
A7.4	Date-filtered loan and overdue reports. Official IT 401W PDF, p.2, A7	TC-A7-004	PASS
A7.5	CSV exports match reports. Official IT 401W PDF, p.2, A7	TC-A7-005	PASS
A7.6	ORM transactions and related/computed totals drive values dynamically. Official IT 401W PDF, p.2, A7	TC-A7-001, TC-A7-006	PASS

Requirement	Requirement and source page	Scenario IDs	Result
A8.1	Approval/rejection/cancellation/release/return/maintenance/account/catalog events retain actor/time/action/reference. Official IT 401W PDF, p.2, A8	TC-A8-001, TC-A8-002	PASS
A8.2	Searchable Admin-only audit history. Official IT 401W PDF, p.2, A8	TC-A8-003	PASS
A8.3	Validated configurable quantity/duration policies apply to new requests. Official IT 401W PDF, p.2, A8	TC-A8-004, TC-A8-005	PASS
U1.1	Django registration/sign-in/sign-out and password handling. Official IT 401W PDF, p.2, U1	TC-U1-001, TC-U1-002, TC-U1-003, TC-U1-004	PASS
U1.2	Only personal name/contact fields editable; no role or activation editing. Official IT 401W PDF, p.2, U1	TC-U1-005, TC-U1-006	PASS
U2.1	Search active equipment by name/category. Official IT 401W PDF, p.2, U2	TC-U2-001, TC-U2-002	PASS
U2.2	Descriptions, total serviceable and available quantities. Official IT 401W PDF, p.2, U2	TC-U2-003, TC-U2-004	PASS
U2.3	Hide identities, internal notes, audits and maintenance details. Official IT 401W PDF, p.2, U2	TC-U2-005	PASS
U3.1	One-to-three different catalog items with positive quantities. Official IT 401W PDF, p.2, U3	TC-U3-001, TC-U3-002, TC-U3-003, TC-U3-004, TC-U3-008	PASS
U3.2	Purpose and desired date obey duration/quantity limits. Official IT 401W PDF, p.2, U3	TC-U3-004, TC-U3-005, TC-U3-006	PASS
U3.3	Submission remains Pending and reserves no physical unit. Official IT 401W PDF, p.2, U3	TC-U3-007	PASS
U4.1	Only own request/loan/history/status/due/released-returned data. Official IT 401W PDF, p.3, U4	TC-U4-001	PASS
U4.2	Cancel Pending/Approved before issue; Approved cancellation frees reservations. Official IT 401W PDF, p.3, U4	TC-U4-002, TC-U4-003	PASS
U4.3	Basic Users cannot approve, issue, record returns or revise due dates. Official IT 401W PDF, p.3, U4	TC-U4-004	PASS
BR1	Exact all-or-nothing atomic reservation and reservation release on Approved cancellation. Official IT 401W PDF, p.4, BR1	TC-A3-006, TC-U4-002, TC-BR1-001, TC-BR1-002	PASS
BR2	Exactly one effective Available/Reserved/Issued/Maintenance/Retired status per unit. Official IT 401W PDF, p.4, BR2	TC-A5-005, TC-A6-006, TC-A6-007, TC-BR2-001, TC-BR2-002	PASS

Requirement	Requirement and source page	Scenario IDs	Result
BR3	Transactions/locking/constraints allow exactly one last-unit competing approval. Official IT 401W PDF, p.4, BR3	TC-BR3-001, TC-BR3-002	PASS
BR4	Unique item per request; validate all rows before saving; no partial invalid request. Official IT 401W PDF, p.4, BR4	TC-U3-001, TC-U3-002, TC-U3-003, TC-BR4-001	PASS
BR5	Default 7 calendar days and 3 units/item; future/in-policy dates; expired issue refused; recorded Admin revision before issue. Official IT 401W PDF, p.4, BR5	TC-A8-004, TC-U3-005, TC-BR5-001, TC-BR5-002, TC-BR5-003, TC-DEMO-002	PASS
BR6	Zero overdue days before/on due date; calendar-date difference only for outstanding loans; no payment/fine/procurement/supplier features required. Official IT 401W PDF, p.4, BR6	TC-A5-004, TC-BR6-001, TC-BR6-002, TC-BR6-003	PASS
RA.1	Exactly two roles and role redirects; prevent self-promotion; authorized Admin/documented setup only. Official IT 401W PDF, p.3, RA	TC-A1-006, TC-U1-003, TC-RA-001	PASS
RA.2	Owner-scope all protected user lookups and deny Admin/AJAX/report/export actions. Official IT 401W PDF, p.3, RA	TC-U1-006, TC-U4-001, TC-U4-004, TC-RA-002	PASS
RA.3	Enforce active-account state at protected server actions. Official IT 401W PDF, p.3, RA	TC-A1-004, TC-A1-005, TC-RA-002	PASS
UI.1	Explicit empty states, useful form errors, confirmations and readable textual statuses. Official IT 401W PDF, p.3, UI	TC-U3-008, TC-UI-001	PASS
UI.2	Consistent navigation and responsive layouts for both roles. Official IT 401W PDF, p.3, UI	TC-UI-002, TC-UI-003	PARTIAL
DB.1	User/Profile identity, role, name/contact and activation data. Official IT 401W PDF, p.3, DB	TC-DB-001	PASS
DB.2	Category/CatalogItem name/description/archive/category relation. Official IT 401W PDF, p.3, DB	TC-A2-005, TC-A2-013, TC-DB-001, TC-DB-006	PASS
DB.3	EquipmentUnit unique code, item, condition, acquisition and status. Official IT 401W PDF, p.3, DB	TC-A2-008, TC-A2-009, TC-DB-001, TC-DB-002, TC-DB-006	PASS
DB.4	BorrowRequest/RequestItem owner/purpose/dates/status/unique item/positive quantity. Official IT 401W PDF, p.3, DB	TC-DB-001, TC-DB-003	PASS
DB.5	UnitAllocation item/unit/timestamp/tracking and unique active unit. Official IT 401W PDF, p.3, DB	TC-DB-001, TC-DB-003	PASS

Requirement	Requirement and source page	Scenario IDs	Result
DB.6	Loan/ReturnRecord one loan/request, issue actor/date and unique per-unit returns. Official IT 401W PDF, p.3, DB	TC-A4-001, TC-A5-001, TC-A5-003, TC-DB-001, TC-DB-004	PASS
DB.7	MaintenanceRecord unit/problem/action/dates/responsible Admin/status. Official IT 401W PDF, p.3, DB	TC-A6-001, TC-DB-001, TC-DB-005	PASS
DB.8	AuditLog/SystemSetting actor/action/time/reference and limits; no Basic User access. Official IT 401W PDF, p.3, DB	TC-A8-001, TC-DB-001, TC-DB-005	PASS
TECH.1	Python/Django ORM, migrations, server templates, inheritance, named URLs, forms and authentication. Official IT 401W PDF, p.4, TECH	TC-TECH-001	PASS
TECH.2	Custom Admin and role bases/reusable components; no generated Admin interface. Official IT 401W PDF, p.4, TECH	TC-TECH-001, TC-TECH-002	PASS
TECH.3	ORM relations/filter/order/pagination/aggregates and reusable transaction logic; at least one related/computed report. Official IT 401W PDF, p.4, TECH	TC-A7-006, TC-TECH-003	PASS
TECH.4	Server validation, CSRF, POST mutations, hashing, escaping, ownership and private secrets. Official IT 401W PDF, p.4, TECH	TC-TECH-004, TC-TECH-005	PASS
TECH.5	Pinned dependencies, environment/migration/seed/README setup; explain SQLite concurrency limits; local demo sufficient. Official IT 401W PDF, p.4, TECH	TC-TECH-006	PASS
DEMO.1	At least 3 categories, 6 items, 18 units, 1 Admin, 8 Basic Users, 20 requests, 8 loans and 4 maintenance records, with rejected/cancelled/partial/damaged/overdue scenarios. Official IT 401W PDF, p.3, DEMO	TC-DEMO-001	PASS
DEMO.2	Fictional identities and repeatable seed; historical fixtures do not weaken ordinary current-date validation (also p.4). Official IT 401W PDF, p.3, DEMO	TC-DEMO-001, TC-DEMO-002	PASS
SUB.1	ZIP contains source/migrations/requirements/README/seed/docs and excludes venv/cache/credentials/private data. Official IT 401W PDF, p.4, SUB	TC-SUB-001	PASS
SUB.2	Privately supply one Admin and two Basic User credentials through the instructor channel. Official IT 401W PDF, p.4, SUB	TC-SUB-002	BLOCKED

Requirement	Requirement and source page	Scenario IDs	Result
EVID.1	User requirements, two options/recommendation, acceptance evidence, resolved feedback and maintenance guidance. Official IT 401W PDF, p.5, EVID	TC-EVID-001	BLOCKED
EVID.2	MTV/database/workflow diagrams, code/tests/limitations and relevant standards/legal/ethical analysis. Official IT 401W PDF, p.5, EVID	TC-EVID-002	PASS
EVID.3	Tool/emerging-technology evaluation and privacy/IP/accessibility/fairness/conflict/public-welfare safeguards and accountability. Official IT 401W PDF, p.5, EVID	TC-EVID-003	BLOCKED
EVID.4	Actual 5-6 members, individual contributions, approximate effort allocation, collaboration, peer feedback and rotated leadership/performance improvements (also p.1). Official IT 401W PDF, p.5, EVID	TC-EVID-004	BLOCKED
EVID.5	Audience-specific guides plus 12-minute demo/3-minute Q&A, identifiable segments and recorded feedback/actions. Official IT 401W PDF, p.5, EVID	TC-EVID-005	NOT EXECUTED
EVID.6	Reused code/assets/AI declaration under instructor/BulSU policy, individual rubric evidence and proponents able to explain their work. Official IT 401W PDF, p.5, EVID	TC-EVID-006	BLOCKED

2.1 Requirements without any verified passing scenario

SUB.2 - Privately supply one Admin and two Basic User credentials through the instructor channel. [BLOCKED]

EVID.1 - User requirements, two options/recommendation, acceptance evidence, resolved feedback and maintenance guidance. [BLOCKED]

EVID.3 - Tool/emerging-technology evaluation and privacy/IP/accessibility/fairness/conflict/public-welfare safeguards and accountability. [BLOCKED]

EVID.4 - Actual 5-6 members, individual contributions, approximate effort allocation, collaboration, peer feedback and rotated leadership/performance improvements (also p.1). [BLOCKED]

EVID.5 - Audience-specific guides plus 12-minute demo/3-minute Q&A, identifiable segments and recorded feedback/actions. [NOT EXECUTED]

EVID.6 - Reused code/assets/AI declaration under instructor/BulSU policy, individual rubric evidence and proponents able to explain their work. [BLOCKED]

2.2 Requirements with mixed verification

A4.2 - Printable slip contains borrower and asset codes. At least one linked case has not passed.

UI.2 - Consistent navigation and responsive layouts for both roles. At least one linked case has not passed.

3. Requirement-derived scenario records

A1 - Accounts

TC-A1-001 | Search and filter registered accounts

Requirement ID: A1.1
Module: Accounts
Test scenario: Search and filter registered accounts

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: Usernames, full names, email and combined account filters

Test steps
1. Sign in as Admin.
2. Search each account attribute and combine filters.
3. Inspect the returned user rows.

Expected result: Only matching accounts appear; records are not changed.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T001

TC-A1-002 | Deactivate then reactivate without data loss

Requirement ID: A1.2
Module: Accounts
Test scenario: Deactivate then reactivate without data loss

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: Existing Basic User with transaction history

Test steps
1. POST a confirmed deactivation as Admin.
2. Verify active status and audit.
3. Reactivate and inspect retained history.

Expected result: Account state changes are audited; borrowing/return history remains.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T002, T003

TC-A1-003 | Inspect a target account history

Requirement ID: A1.3
Module: Accounts
Test scenario: Inspect a target account history

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: Two borrowers with different requests, loans and returns

Test steps
1. Open one Admin account detail.
2. Inspect its request and loan history.
3. Compare rows with the target identity.

Expected result: The selected account history is shown without mixing another account.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T004

TC-A1-004 | Deny new sign-in for inactive account

Requirement ID: A1.4, RA.3
Module: Accounts
Test scenario: Deny new sign-in for inactive account

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: Correct credentials on an inactive account

Test steps
1. Deactivate the fixture account.
2. Attempt sign-in with its valid credentials.
3. Inspect authentication state.

Expected result: The inactive account is not authenticated.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T005

TC-A1-005 | Deactivate an already authenticated session

Requirement ID: A1.4, RA.3
Module: Accounts
Test scenario: Deactivate an already authenticated session

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: A signed-in Basic User deactivated by Admin

Test steps
1. Establish a Basic User session.
2. Deactivate its account.
3. Request protected pages/transactions with that session.

Expected result: Existing sessions lose protected access and cannot transact.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T006, T007

TC-A1-006 | Reject registration self-promotion

Requirement ID: A1.5, RA.1
Module: Accounts
Test scenario: Reject registration self-promotion

Preconditions: Isolated migrated fixture; active Admin and Basic User accounts; target history where stated. Use fresh fixtures per scenario.

Test data: Forged role, staff/superuser and permission inputs

Test steps
1. POST registration with privilege fields.
2. Reload the created account.
3. Inspect custom role and permissions.

Expected result: Registration creates only an ordinary Basic User and never grants Admin access.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T008

A2 - Catalog and physical units

TC-A2-001 | Create and update a category

Requirement ID: A2.1
Module: Catalog and physical units
Test scenario: Create and update a category

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Valid name, surrounding spaces and optional description

Test steps
1. Use the Admin category service/form.
2. Create and edit a category.
3. Inspect normalized name and saved description.

Expected result: Category is created/updated through validated custom operations.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T009, T010

TC-A2-002 | Archive and restore category relationships

Requirement ID: A2.1
Module: Catalog and physical units
Test scenario: Archive and restore category relationships

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Category already referenced by equipment and borrowing

Test steps
1. Archive the referenced category.
2. Inspect linked equipment and stock.
3. Restore the category.

Expected result: Archive/restore retains records, relationships and stock rather than deleting history.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T011

TC-A2-003 | Prevent new assignment to archived category

Requirement ID: A2.1
Module: Catalog and physical units
Test scenario: Prevent new assignment to archived category

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Archived category; new or differently categorized item

Test steps
1. Attempt new catalog creation using the archive.
2. Attempt reassignment of another item.
3. Inspect records after rejection.

Expected result: New assignments to an archived category are rejected; existing relationships remain valid.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T012, T013

TC-A2-004 | Reject duplicate category names including archived names

Requirement ID: A2.1
Module: Catalog and physical units
Test scenario: Reject duplicate category names including archived names

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Case/space variants and archived category name

Test steps
1. Create a category.
2. Attempt equivalent names through validation/database.
3. Inspect failed write and original row.

Expected result: Equivalent names are rejected and an edit does not implicitly restore an archive.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T014, T015

TC-A2-005 | Create/edit catalog item with valid category

Requirement ID: A2.2, DB.2
Module: Catalog and physical units
Test scenario: Create/edit catalog item with valid category

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Name, description, existing category; forged stock/state fields

Test steps
1. Submit catalog create/edit as Admin.
2. Include forged derived stock/state values.
3. Inspect catalog and physical-unit rows.

Expected result: Only permitted metadata is saved; stock remains derived and category is validated.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T016, T017

TC-A2-006 | Archive/restore catalog without removing units

Requirement ID: A2.2, A2.6
Module: Catalog and physical units
Test scenario: Archive/restore catalog without removing units

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Catalog with physical units and retained history

Test steps
1. Archive the eligible catalog.
2. Inspect units/status/history.
3. Restore the master record.

Expected result: Catalog active state changes without deleting units or rewriting their status.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T018

TC-A2-007 | Reject archiving stock with live reservations/issues

Requirement ID: A2.2
Module: Catalog and physical units
Test scenario: Reject archiving stock with live reservations/issues

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Reserved and Issued physical units

Test steps
1. Prepare active borrowing relationships.
2. Attempt catalog archive.
3. Verify the master and allocations remain unchanged.

Expected result: The transaction-linked item cannot be archived through a bypass.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T019

TC-A2-008 | Register and normalize a unique physical asset code

Requirement ID: A2.3, DB.3
Module: Catalog and physical units
Test scenario: Register and normalize a unique physical asset code

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Valid code, case/space duplicates, empty or overlong code

Test steps
1. Create a physical unit.
2. Attempt duplicate/invalid identifiers.
3. Inspect saved identifier and unit count.

Expected result: Valid identifier is normalized; duplicate and invalid codes are refused.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T020, T021, T022

TC-A2-009 | Store condition and acquisition date separately from status

Requirement ID: A2.4, DB.3
Module: Catalog and physical units
Test scenario: Store condition and acquisition date separately from status

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Good/Fair/Poor/Damaged and valid acquisition date

Test steps
1. Create/edit unit metadata.
2. Reload each physical unit.
3. Compare metadata with inventory status.

Expected result: Supported condition/date values are retained without automatic inventory-status changes.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T023

TC-A2-010 | Reject invalid condition and future acquisition date

Requirement ID: A2.4
Module: Catalog and physical units
Test scenario: Reject invalid condition and future acquisition date

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Unsupported condition, future date and invalid date

Test steps
1. Submit invalid metadata via form/service.
2. Attempt invalid condition at database boundary.
3. Verify original values remain.

Expected result: Invalid values fail validation and do not partially overwrite the unit.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T024, T025

TC-A2-011 | Combine physical-unit filters

Requirement ID: A2.5
Module: Catalog and physical units
Test scenario: Combine physical-unit filters

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Category, equipment, status and search inputs

Test steps
1. Apply each filter alone.
2. Apply category/item/status/search together.
3. Inspect each resulting unit relationship.

Expected result: Every displayed unit matches the combined requested conditions.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T026

TC-A2-012 | Retain filters across pages and reset cleanly

Requirement ID: A2.5
Module: Catalog and physical units
Test scenario: Retain filters across pages and reset cleanly

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: A multipage matching unit list

Test steps
1. Navigate the next filtered page.
2. Inspect query parameters and rows.
3. Use Reset filters.

Expected result: Pagination preserves filters; reset clears them and restores defaults.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T027

TC-A2-013 | Protect referenced category deletion

Requirement ID: A2.6, DB.2
Module: Catalog and physical units
Test scenario: Protect referenced category deletion

Preconditions: Isolated inventory fixture; active Admin; referenced categories/items/units and statuses as specified. Preserve baseline outside rollback tests.

Test data: Referenced category, including archived equipment

Test steps
1. Attempt deletion through service and HTTP.
2. Inspect related records after rejection.
3. Check the original category remains.

Expected result: Protected relationships preserve transaction and inventory history.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T028, T029

A3 - Request review

TC-A3-001 | Search/filter/page the request queue

Requirement ID: A3.1
Module: Request review
Test scenario: Search/filter/page the request queue

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Pending/all statuses; borrower/equipment/purpose terms

Test steps
1. Sign in as Admin.
2. Apply queue search/status filters.
3. Navigate filtered result pages.

Expected result: Queue rows reflect filters and pagination retains their parameters.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T030, T031

TC-A3-002 | Use inclusive Manila date boundaries and reject reversed dates

Requirement ID: A3.1
Module: Request review
Test scenario: Use inclusive Manila date boundaries and reject reversed dates

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Start/end local dates with midnight boundary fixtures

Test steps
1. Apply combined search/status/date filters.
2. Inspect end-date inclusivity and next-midnight exclusion.
3. Submit invalid/reversed date ranges.

Expected result: Valid boundaries work with pagination; invalid ranges show errors rather than misleading results.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T032, T033

TC-A3-003 | Review current availability without reserving stock

Requirement ID: A3.2
Module: Request review
Test scenario: Review current availability without reserving stock

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Mixed physical statuses and a Pending request

Test steps
1. GET request/review pages.
2. Change fixture unit eligibility and reload review.
3. Inspect current available counts and allocations.

Expected result: Review reflects live eligible stock; GET does not change workflow or reserve units.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T034, T035

TC-A3-004 | Require and persist approval reason

Requirement ID: A3.3
Module: Request review
Test scenario: Require and persist approval reason

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Valid reason and blank reason; active Admin

Test steps
1. Attempt approval with blank reason.
2. Approve with a valid reason.
3. Inspect decision, actor, timestamp and audit.

Expected result: Blank rationale is rejected; successful reason is saved, escaped and audited.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T036, T037

TC-A3-005 | Reject with recorded reason

Requirement ID: A3.3
Module: Request review
Test scenario: Reject with recorded reason

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Pending request; valid, blank and forged rejection inputs

Test steps
1. Attempt rejection without required rationale.
2. Reject as Admin with a valid reason.
3. Inspect borrower-visible decision history.

Expected result: Rejection requires a reason and records the responsible Admin/date; unsafe HTML is escaped.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T038, T039

TC-A3-006 | Reserve exact quantities on approval

Requirement ID: A3.4, BR1
Module: Request review
Test scenario: Reserve exact quantities on approval

Preconditions: Isolated request/stock fixtures; active Admin; Pending request unless otherwise stated; known live policy.

Test data: Eligible Available units for each requested quantity

Test steps
1. Submit a Pending request.
2. Approve through the allocation service.
3. Count specific active allocations and unit statuses.

Expected result: Exactly the requested units become Reserved and the approval actor/time are recorded.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T040, T041

A4 - Equipment release

TC-A4-001 | Issue Approved request with immutable handover data

Requirement ID: A4.1, DB.6
Module: Equipment release
Test scenario: Issue Approved request with immutable handover data

Preconditions: Isolated Approved request with approval metadata and exact active allocations unless testing an invalid precondition; active Admin.

Test data: Complete valid approval and reserved-unit allocation

Test steps
1. Invoke release as Admin.
2. Inspect Loan and LoanUnit rows.
3. Compare due date, actor, unit identifiers and snapshots.

Expected result: One loan is created with the final agreed due date, release Admin/time and exact unit snapshots.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T042

TC-A4-002 | Render borrowing slip with borrower and asset codes

Requirement ID: A4.2
Module: Equipment release
Test scenario: Render borrowing slip with borrower and asset codes

Preconditions: Isolated Approved request with approval metadata and exact active allocations unless testing an invalid precondition; active Admin.

Test data: An existing issued loan and its assigned units

Test steps
1. Open the Admin printable slip.
2. Inspect borrower/issue/due information.
3. Check the actual unit asset codes and print control.

Expected result: The slip contains the required handover details without granting Basic User print access.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T043, T044

TC-A4-003 | Check physical print output

Requirement ID: A4.2
Module: Equipment release
Test scenario: Check physical print output

Preconditions: Isolated Approved request with approval metadata and exact active allocations unless testing an invalid precondition; active Admin.

Test data: Printer or browser print-preview review required

Test steps
1. Open the borrowing slip.
2. Activate Print borrowing slip.
3. Inspect paper size, pagination and actual printed output.

Expected result: Borrower and asset codes remain readable on physical output.

Actual result: Physical print-preview/paper inspection was not performed; screenshot 23 proves only the rendered web slip.

Status: NOT EXECUTED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

TC-A4-004 | Reject invalid and duplicate release

Requirement ID: A4.3
Module: Equipment release
Test scenario: Reject invalid and duplicate release

Preconditions: Isolated Approved request with approval metadata and exact active allocations unless testing an invalid precondition; active Admin.

Test data: Pending/Rejected/Cancelled/already-issued requests

Test steps
1. Attempt release for each non-Approved request.
2. Release a valid approval once.
3. Repeat release for the issued request.

Expected result: Invalid states are rejected; no duplicate loan or issue is created.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T045, T046

TC-A4-005 | Reject incomplete/wrong/unavailable allocations atomically

Requirement ID: A4.3
Module: Equipment release
Test scenario: Reject incomplete/wrong/unavailable allocations atomically

Preconditions: Isolated Approved request with approval metadata and exact active allocations unless testing an invalid precondition; active Admin.

Test data: Missing units, wrong equipment and one unavailable allocated unit

Test steps
1. Prepare each invalid allocation fixture.
2. Attempt release.
3. Inspect request/unit/loan state after rejection.

Expected result: Invalid release creates no loan and leaves no partial unit transition.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T047, T048, T049

A5 - Returns and overdue

TC-A5-001 | Record a partial return per physical unit

Requirement ID: A5.1, A5.2, DB.6
Module: Returns and overdue
Test scenario: Record a partial return per physical unit

Preconditions: Isolated issued loan with known outstanding LoanUnit rows; authorized receiving Admin; each scenario starts from its defined fixture.

Test data: One selected unit from a multi-unit issued loan

Test steps
1. Return one chosen unit as Good.
2. Inspect its return date/condition/Admin.
3. Count still-outstanding units.

Expected result: Only the selected unit is returned; remaining units stay issued and the loan stays open.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T050

TC-A5-002 | Complete only on the final return

Requirement ID: A5.2
Module: Returns and overdue
Test scenario: Complete only on the final return

Preconditions: Isolated issued loan with known outstanding LoanUnit rows; authorized receiving Admin; each scenario starts from its defined fixture.

Test data: Loan with some units already returned

Test steps
1. Return the remaining units.
2. Reload the loan and request.
3. Inspect completion and all unit histories.

Expected result: The final return completes the loan/request without rewriting earlier return records.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T051, T052

TC-A5-003 | Prevent duplicate and cross-loan returns

Requirement ID: A5.1, DB.6
Module: Returns and overdue
Test scenario: Prevent duplicate and cross-loan returns

Preconditions: Isolated issued loan with known outstanding LoanUnit rows; authorized receiving Admin; each scenario starts from its defined fixture.

Test data: Returned unit; unit belonging to a different loan

Test steps
1. Attempt duplicate return with new notes.
2. Attempt to return another loan unit.
3. Compare retained return records.

Expected result: Both invalid returns are rejected; existing history is not overwritten.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T053, T054

TC-A5-004 | Remove completed/zero-outstanding loans from overdue results

Requirement ID: A5.3, BR6
Module: Returns and overdue
Test scenario: Remove completed/zero-outstanding loans from overdue results

Preconditions: Isolated issued loan with known outstanding LoanUnit rows; authorized receiving Admin; each scenario starts from its defined fixture.

Test data: Overdue partially returned loan, then final return

Test steps
1. Evaluate overdue state with an outstanding unit.
2. Record the last return.
3. Re-evaluate day count, dashboard/report/query filters.

Expected result: Partial outstanding debt remains overdue; fully returned loans have zero overdue days and no overdue classification.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T055, T056

TC-A5-005 | Send a damaged return to Maintenance

Requirement ID: A5.4, BR2
Module: Returns and overdue
Test scenario: Send a damaged return to Maintenance

Preconditions: Isolated issued loan with known outstanding LoanUnit rows; authorized receiving Admin; each scenario starts from its defined fixture.

Test data: Issued unit returned Damaged with required notes

Test steps
1. Record the damaged return.
2. Inspect physical status and linked maintenance.
3. Inspect original loan/return history.

Expected result: The unit is not made Available; a linked maintenance record is created atomically.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T057, T058

A6 - Maintenance and reconciliation

TC-A6-001 | Open maintenance with problem, Admin and date

Requirement ID: A6.1, DB.7
Module: Maintenance and reconciliation
Test scenario: Open maintenance with problem, Admin and date

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: Idle eligible physical unit and nonblank issue

Test steps
1. Create a maintenance report as Admin.
2. Reload unit and record.
3. Inspect initial status, reporter and timestamp.

Expected result: New report is Reported and the physical unit is in Maintenance.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T059

TC-A6-002 | Start repair without losing report history

Requirement ID: A6.1
Module: Maintenance and reconciliation
Test scenario: Start repair without losing report history

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: Reported record with original Admin/notes

Test steps
1. Confirm Start repair as a permitted Admin.
2. Inspect status and physical inventory.
3. Read original reporting data and new audit actor/time.

Expected result: Record becomes In Progress; original report data is retained and unit remains Maintenance.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T060

TC-A6-003 | Complete a serviceable repair

Requirement ID: A6.2
Module: Maintenance and reconciliation
Test scenario: Complete a serviceable repair

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: In Progress record with repair action notes

Test steps
1. Complete repair as Repaired.
2. Inspect completion Admin/date/action.
3. Inspect physical-unit availability.

Expected result: Repair is recorded and the unit returns to Available.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T061

TC-A6-004 | Retire an irreparable unit and preserve closed history

Requirement ID: A6.2
Module: Maintenance and reconciliation
Test scenario: Retire an irreparable unit and preserve closed history

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: In Progress maintenance record; Retired outcome

Test steps
1. Complete repair with a retirement outcome.
2. Attempt to restart/edit closed work.
3. Inspect inventory eligibility and history.

Expected result: Unit stays unavailable/Retired and closed history cannot be restarted or overwritten.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T062

TC-A6-005 | Reject invalid or reversed maintenance transitions

Requirement ID: A6.1, A6.2
Module: Maintenance and reconciliation
Test scenario: Reject invalid or reversed maintenance transitions

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: Blank resolution, unsupported outcome or completed record

Test steps
1. Attempt completion without required notes/progress/confirmation.
2. Attempt backward or repeated transition.
3. Inspect record/unit/audit unchanged state.

Expected result: Invalid transitions fail server-side without partial state changes.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T063, T064

TC-A6-006 | Refuse maintenance on Reserved and Issued units

Requirement ID: A6.3, BR2
Module: Maintenance and reconciliation
Test scenario: Refuse maintenance on Reserved and Issued units

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: Reserved/ON_LOAN inventory and active relationships

Test steps
1. Try new report in the form/service for each busy unit.
2. Inspect records and unit statuses.
3. Repeat against stale/inconsistent availability.

Expected result: Busy units cannot be taken into idle-unit maintenance; no new report is saved.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T065, T066

TC-A6-007 | Reconcile derived five-status inventory counts

Requirement ID: A6.4, BR2
Module: Maintenance and reconciliation
Test scenario: Reconcile derived five-status inventory counts

Preconditions: Isolated physical-unit/maintenance fixture and active Admin; status/outcome defined by the test data.

Test data: Mixed Available/Reserved/ON_LOAN/Maintenance/Retired units

Test steps
1. Query inventory aggregates.
2. Perform approval/issue/good/damaged return transitions.
3. Compare each status count with actual physical rows.

Expected result: Derived counts reconcile and change with physical-unit transactions.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T067, T068

A7 - Dashboard and reports

TC-A7-001 | Compute all required live dashboard cards

Requirement ID: A7.1, A7.6
Module: Dashboard and reports
Test scenario: Compute all required live dashboard cards

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Physical stock, Pending requests and outstanding loans

Test steps
1. Load the Admin dashboard.
2. Compare card values with fixture records.
3. Create/change request state and reload.

Expected result: Available/Pending/Active/Overdue values derive from current records and update dynamically.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T069, T070

TC-A7-002 | Count loans by issued month rather than request rows

Requirement ID: A7.2
Module: Dashboard and reports
Test scenario: Count loans by issued month rather than request rows

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Requests, multi-unit loans and month-boundary dates

Test steps
1. Query the required chart period.
2. Group actual issued Loan rows by local month.
3. Compare totals with the rendered series.

Expected result: Each issued loan counts once in its issue month; requests/units do not multiply loan totals.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T071

TC-A7-003 | Count issued equipment usage by category

Requirement ID: A7.3
Module: Dashboard and reports
Test scenario: Count issued equipment usage by category

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Issued units in several categories, including later returns

Test steps
1. Query category usage for a date period.
2. Return some units.
3. Compare category totals with issuance rows.

Expected result: Each unit issuance is counted by category; completed returns remain historical usage.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T072

TC-A7-004 | Filter loan and overdue reports with the correct date field

Requirement ID: A7.4
Module: Dashboard and reports
Test scenario: Filter loan and overdue reports with the correct date field

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Inclusive issue/due dates and local calendar boundaries

Test steps
1. Filter the loan report by issue date.
2. Filter overdue report by due date.
3. Check inclusive endpoints and next-midnight exclusion.

Expected result: Loan and overdue reports use their defined date fields and local boundaries consistently.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T073, T074, T075

TC-A7-005 | Export every matching report row safely

Requirement ID: A7.5
Module: Dashboard and reports
Test scenario: Export every matching report row safely

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Multipage matching data; commas/formula-like user values

Test steps
1. Apply report filters.
2. Export CSV.
3. Inspect full filtered rows, quoting, headers and sensitive-note exclusion.

Expected result: CSV matches all filtered records, not only one page, and escapes risky spreadsheet values.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T076, T077, T078

TC-A7-006 | Combine related models and computed outstanding totals

Requirement ID: A7.6, TECH.3
Module: Dashboard and reports
Test scenario: Combine related models and computed outstanding totals

Preconditions: Isolated historical/current records, known local dates, active Admin and report access.

Test data: Loan with partial returns and renamed inventory metadata

Test steps
1. Build related-model report rows.
2. Inspect returned/outstanding totals and saved issuance labels.
3. Compare display with transaction records.

Expected result: The report combines real related data and computed totals while preserving issue snapshots.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T079, T080

A8 - Audit and settings

TC-A8-001 | Audit required workflow and inventory events

Requirement ID: A8.1, DB.8
Module: Audit and settings
Test scenario: Audit required workflow and inventory events

Preconditions: Isolated policy/audit fixtures and active Admin; event failures are injected only inside test transactions.

Test data: Successful decisions, release, returns and maintenance

Test steps
1. Perform each supported fixture transition.
2. Read AuditLog entries.
3. Check action, actor, timestamp, reference and nonduplication.

Expected result: Distinct required events are attributable and reference their records.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T081, T082, T083

TC-A8-002 | Roll back business writes when audit recording fails

Requirement ID: A8.1
Module: Audit and settings
Test scenario: Roll back business writes when audit recording fails

Preconditions: Isolated policy/audit fixtures and active Admin; event failures are injected only inside test transactions.

Test data: Injected audit failure during a state-changing service

Test steps
1. Inject the audit write error.
2. Attempt the transaction.
3. Inspect business rows and audit entries afterward.

Expected result: The transaction rolls back instead of leaving unaudited partial changes.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T084, T085

TC-A8-003 | Search and protect the audit history

Requirement ID: A8.2
Module: Audit and settings
Test scenario: Search and protect the audit history

Preconditions: Isolated policy/audit fixtures and active Admin; event failures are injected only inside test transactions.

Test data: Actor/action/entity/record/date inputs; Basic User session

Test steps
1. Search the Admin audit list.
2. Inspect filtered newest-first rows.
3. Attempt access/export from Basic User context.

Expected result: Audit data is searchable and restricted to authorized Admin accounts.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T086, T087, T088

TC-A8-004 | Use defaults and validate revised operational limits

Requirement ID: A8.3, BR5
Module: Audit and settings
Test scenario: Use defaults and validate revised operational limits

Preconditions: Isolated policy/audit fixtures and active Admin; event failures are injected only inside test transactions.

Test data: Default 3 items/3 units/7 days; invalid numeric/range inputs

Test steps
1. Read policy without a saved row.
2. Submit valid and invalid limit updates.
3. Inspect saved values and audit/rollback behavior.

Expected result: Defaults apply; invalid settings fail validation without partial policy writes.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T089, T090, T091

TC-A8-005 | Apply new policy to new requests, preserve recorded agreements

Requirement ID: A8.3
Module: Audit and settings
Test scenario: Apply new policy to new requests, preserve recorded agreements

Preconditions: Isolated policy/audit fixtures and active Admin; event failures are injected only inside test transactions.

Test data: Changed quantity/duration; previously Approved or Issued records

Test steps
1. Save revised limits.
2. Submit a new request.
3. Release an existing agreement and inspect prior loan terms.

Expected result: New requests obey current policy; changes do not silently rewrite Approved/Issued terms.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T092, T093, T094

U1 - Authentication and profile

TC-U1-001 | Register and store a secure Django password

Requirement ID: U1.1
Module: Authentication and profile
Test scenario: Register and store a secure Django password

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: Unique fictional username/email and policy-compliant password

Test steps
1. POST valid registration.
2. Inspect the new role and authenticated session.
3. Check the stored password with Django password verification.

Expected result: An active Basic User is created; the password is hashed rather than stored in plaintext.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T095, T096

TC-U1-002 | Reject invalid registration and authentication

Requirement ID: U1.1
Module: Authentication and profile
Test scenario: Reject invalid registration and authentication

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: Duplicate username, malformed contact/email, weak/mismatched password and wrong credentials

Test steps
1. Submit each invalid registration input.
2. Attempt sign-in with wrong credentials.
3. Inspect errors, account count and authentication.

Expected result: Invalid data is rejected with no unintended account/session.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T097, T098, T099, T100

TC-U1-003 | Redirect valid sign-in by campus role

Requirement ID: U1.1, RA.1
Module: Authentication and profile
Test scenario: Redirect valid sign-in by campus role

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: One Admin and one Basic User

Test steps
1. Sign in with each account.
2. Inspect response destination.
3. Attempt an unsupported legacy role.

Expected result: Admin enters /manage/; Basic User enters /portal/; unsupported roles are denied.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T101, T102

TC-U1-004 | Sign out with a CSRF-protected POST

Requirement ID: U1.1
Module: Authentication and profile
Test scenario: Sign out with a CSRF-protected POST

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: Authenticated session and valid/invalid CSRF tokens

Test steps
1. Attempt GET logout.
2. POST logout with CSRF protection.
3. Inspect the former session and protected-page response.

Expected result: GET does not sign out; valid POST flushes the session and CSRF remains enforced.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T103, T104

TC-U1-005 | Update only permitted personal information

Requirement ID: U1.2
Module: Authentication and profile
Test scenario: Update only permitted personal information

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: Name/contact updates with forged role/activation fields

Test steps
1. POST personal profile changes plus privileged fields.
2. Reload the account.
3. Inspect saved values and role/active state.

Expected result: Permitted personal fields update; role, activation and other privileged fields cannot be changed.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T105, T106

TC-U1-006 | Prevent cross-account profile access and stale privilege overwrite

Requirement ID: U1.2, RA.2
Module: Authentication and profile
Test scenario: Prevent cross-account profile access and stale privilege overwrite

Preconditions: Isolated accounts and valid CSRF configuration; no real personal data; sign-in state set by the scenario.

Test data: Other user identifier and stale account instance

Test steps
1. Try another-user profile URL.
2. Try Admin profile aliases as Basic User.
3. Save a stale profile after Admin role/activation change.

Expected result: No other-user profile endpoint is exposed and stale profile data cannot restore old privileges.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T107, T108, T109, T110

U2 - Equipment browsing

TC-U2-001 | Combine active-catalog name/category/availability filters

Requirement ID: U2.1
Module: Equipment browsing
Test scenario: Combine active-catalog name/category/availability filters

Preconditions: Active Basic User, isolated categorized equipment with known stock/archive states, catalog/detail access.

Test data: Several categories, matching/nonmatching names and archived item

Test steps
1. Search by name.
2. Combine category and availability.
3. Navigate pagination and inspect retained query parameters.

Expected result: Only matching active equipment appears and filters survive pagination.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T111, T112

TC-U2-002 | Hide archived catalog items and their direct detail URLs

Requirement ID: U2.1
Module: Equipment browsing
Test scenario: Hide archived catalog items and their direct detail URLs

Preconditions: Active Basic User, isolated categorized equipment with known stock/archive states, catalog/detail access.

Test data: Archived item plus an otherwise empty active catalog

Test steps
1. Load the catalog.
2. Request the archived item detail directly.
3. Inspect the empty result state.

Expected result: Archived items are absent and direct access returns 404.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T113, T114

TC-U2-003 | Display descriptions and mixed-status serviceable totals

Requirement ID: U2.2
Module: Equipment browsing
Test scenario: Display descriptions and mixed-status serviceable totals

Preconditions: Active Basic User, isolated categorized equipment with known stock/archive states, catalog/detail access.

Test data: Available/Reserved/Issued/Maintenance/Retired units and zero stock

Test steps
1. Load catalog and detail pages.
2. Compare Available and Total Serviceable with physical rows.
3. Repeat with zero stock and long/escaped text.

Expected result: Serviceable equals Available+Reserved+Issued; Maintenance/Retired are excluded; text and zero values remain readable.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T115, T116, T117

TC-U2-004 | Refresh public totals through the lending lifecycle

Requirement ID: U2.2
Module: Equipment browsing
Test scenario: Refresh public totals through the lending lifecycle

Preconditions: Active Basic User, isolated categorized equipment with known stock/archive states, catalog/detail access.

Test data: One item traversing reservation, issue, return and maintenance

Test steps
1. Read public totals.
2. Approve, issue and return fixture units.
3. Repair/retire as applicable and read totals again.

Expected result: Counts reflect physical records at each transition without duplicated stored stock totals.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T118

TC-U2-005 | Protect internal inventory and repair information

Requirement ID: U2.3
Module: Equipment browsing
Test scenario: Protect internal inventory and repair information

Preconditions: Active Basic User, isolated categorized equipment with known stock/archive states, catalog/detail access.

Test data: Internal identifiers/notes and Basic User session

Test steps
1. Open public catalog/detail.
2. Inspect rendered content.
3. Attempt maintenance and Admin endpoints as Basic User.

Expected result: Asset identities, internal notes and administrative actions do not leak into public equipment browsing.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T119, T120

U3 - Request submission

TC-U3-001 | Submit one, two or three distinct items

Requirement ID: U3.1, BR4
Module: Request submission
Test scenario: Submit one, two or three distinct items

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Available items; one/two/three valid rows

Test steps
1. Authenticate a Basic User.
2. POST each valid row count with positive quantities.
3. Inspect request and item rows.

Expected result: All three permitted sizes succeed; exactly three is not mandatory.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T121, T122

TC-U3-002 | Reject duplicates, empty/four-item forms and forged management data

Requirement ID: U3.1, BR4
Module: Request submission
Test scenario: Reject duplicates, empty/four-item forms and forged management data

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Duplicate item; no items; four items; invalid formset management

Test steps
1. POST each invalid selection.
2. Inspect validation and request/item/allocation counts.
3. Repeat with forged client step state.

Expected result: The entire invalid request is rejected; no partial rows or allocation is saved.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T123, T124, T125

TC-U3-003 | Cap distinct items despite manipulated policy

Requirement ID: U3.1, BR4
Module: Request submission
Test scenario: Cap distinct items despite manipulated policy

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Configured/legacy max_items greater than three

Test steps
1. Manipulate policy in isolated fixtures.
2. Attempt a four-item service and HTTP submission.
3. Inspect Admin settings validation.

Expected result: The hard maximum remains three and invalid settings cannot raise it.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T126, T127, T128, T129

TC-U3-004 | Reject nonpositive/excessive quantity and blank purpose

Requirement ID: U3.1, U3.2
Module: Request submission
Test scenario: Reject nonpositive/excessive quantity and blank purpose

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Zero, negative, nonnumeric or over-policy quantity; blank purpose

Test steps
1. Submit each invalid value.
2. Check all row errors.
3. Inspect the database for partial requests.

Expected result: Quantity must be positive and within policy; purpose is required; invalid data is not saved.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T130, T131, T132

TC-U3-005 | Validate future date and calendar-day duration

Requirement ID: U3.2, BR5
Module: Request submission
Test scenario: Validate future date and calendar-day duration

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Past/current timestamp, naive datetime, seven-day edge and over-limit date

Test steps
1. Submit boundary due dates.
2. Repeat with revised policy.
3. Inspect errors and stored date for accepted cases.

Expected result: Due date is future and within the configured local-calendar duration.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T133, T134, T135

TC-U3-006 | Refuse unavailable equipment and stale quantities

Requirement ID: U3.2
Module: Request submission
Test scenario: Refuse unavailable equipment and stale quantities

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Zero availability, archived item and insufficient current stock

Test steps
1. Select/forge unavailable equipment.
2. Reduce availability before submission.
3. Submit and inspect record counts.

Expected result: Unavailable selection is disabled in UI and rejected server-side with no partial request.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T136, T137

TC-U3-007 | Keep submitted requests Pending without reservation

Requirement ID: U3.3
Module: Request submission
Test scenario: Keep submitted requests Pending without reservation

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Valid one/two/three-row payload with forged status/borrower fields

Test steps
1. POST valid request data plus forged protected fields.
2. Inspect owner/status and units.
3. Compare allocation count.

Expected result: Server assigns the authenticated owner and Pending state; no physical units are reserved.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T138, T139

TC-U3-008 | Preserve selection and entered data on invalid submission

Requirement ID: U3.1, UI.1
Module: Request submission
Test scenario: Preserve selection and entered data on invalid submission

Preconditions: Active Basic User; isolated available catalog records; valid baseline purpose/due date; known policy limits.

Test data: Preselected valid item, invalid/nonexistent/unavailable IDs and invalid form values

Test steps
1. Open catalog Request equipment link and direct request URL.
2. Try invalid preselection.
3. POST invalid details and inspect redisplayed values.

Expected result: Valid preselection is retained; invalid IDs cannot create selections; validation retains entered values.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T140, T141, T142, T143

U4 - Tracking and cancellation

TC-U4-001 | Scope request, loan, history and dashboard data to owner

Requirement ID: U4.1, RA.2
Module: Tracking and cancellation
Test scenario: Scope request, loan, history and dashboard data to owner

Preconditions: Two isolated borrowers with distinct requests/loans/history; authorized session for the tested borrower.

Test data: Two borrowers with separate requests/loans/history

Test steps
1. Sign in as the first borrower.
2. Search/list/open own and other-user records.
3. Inspect dashboard totals and completed history.

Expected result: Only own records and personal counts appear; cross-owner detail actions are refused.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T144, T145, T146, T147

TC-U4-002 | Cancel Pending without allocating and Approved with full release

Requirement ID: U4.2, BR1
Module: Tracking and cancellation
Test scenario: Cancel Pending without allocating and Approved with full release

Preconditions: Two isolated borrowers with distinct requests/loans/history; authorized session for the tested borrower.

Test data: Owned Pending and Approved requests

Test steps
1. Cancel Pending.
2. Cancel Approved before issue.
3. Inspect cancellation metadata, history, reservations and inventory.

Expected result: Pending cancellation does not reserve; Approved cancellation releases every reservation and retains history.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T148, T149

TC-U4-003 | Refuse foreign, terminal and Issued cancellation

Requirement ID: U4.2
Module: Tracking and cancellation
Test scenario: Refuse foreign, terminal and Issued cancellation

Preconditions: Two isolated borrowers with distinct requests/loans/history; authorized session for the tested borrower.

Test data: Other owner; Issued/Completed/Cancelled request

Test steps
1. Attempt each cancellation through service/HTTP.
2. Inspect states and allocations.
3. Try issue of a cancelled request.

Expected result: Invalid cancellation or release cannot change terminal/issued data or another owner record.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T150, T151, T152

TC-U4-004 | Deny Admin lending and revision actions to Basic Users

Requirement ID: U4.3, RA.2
Module: Tracking and cancellation
Test scenario: Deny Admin lending and revision actions to Basic Users

Preconditions: Two isolated borrowers with distinct requests/loans/history; authorized session for the tested borrower.

Test data: Basic User even with staff/superuser flag

Test steps
1. Attempt review, approval, release, return, slip and due revision routes.
2. Inspect responses and database.
3. Repeat with forged actor fields.

Expected result: The campus role governs access; unauthorized lending/revision writes are denied.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T153, T044, T154

BR1 - Atomic reservation

TC-BR1-001 | Reserve every requested unit atomically or none

Requirement ID: BR1
Module: Atomic reservation
Test scenario: Reserve every requested unit atomically or none

Preconditions: Isolated transactional borrowing fixture with available/competing units; separate connections for concurrency cases.

Test data: Multi-item request with one unavailable final unit

Test steps
1. Attempt approval after availability changes.
2. Inspect every request item/allocation/status.
3. Repeat with valid exact stock.

Expected result: Failure rolls back every reservation; valid approval reserves exactly the required physical units.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T155, T156, T040

TC-BR1-002 | Resolve cancellation/issue races without partial release

Requirement ID: BR1
Module: Atomic reservation
Test scenario: Resolve cancellation/issue races without partial release

Preconditions: Isolated transactional borrowing fixture with available/competing units; separate connections for concurrency cases.

Test data: Approved request and two concurrent actors

Test steps
1. Use separate database connections to race cancel and issue.
2. Inspect winning state.
3. Verify allocation and unit/loan consistency.

Expected result: Exactly one compatible transition wins; no partial release or duplicate issue remains.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T157

BR2 - Physical-unit status

TC-BR2-001 | Exclude every non-Available effective status from allocation

Requirement ID: BR2
Module: Physical-unit status
Test scenario: Exclude every non-Available effective status from allocation

Preconditions: Isolated mixed-status physical-unit fixture; valid actor and lending/maintenance relationships.

Test data: Mixed stock and transaction-linked units

Test steps
1. Request approval against mixed inventory.
2. Inspect selected unit IDs.
3. Try maintenance against Reserved/Issued units.

Expected result: Only eligible Available units are borrowed; live reservations/issues cannot enter maintenance.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T158, T065

TC-BR2-002 | Preserve inventory status through complete lifecycle

Requirement ID: BR2
Module: Physical-unit status
Test scenario: Preserve inventory status through complete lifecycle

Preconditions: Isolated mixed-status physical-unit fixture; valid actor and lending/maintenance relationships.

Test data: Approved item, release, partial return and damaged return

Test steps
1. Execute approval to release.
2. Return one good and one damaged unit.
3. Compare physical status with allocation/loan/maintenance rows.

Expected result: Reserved becomes Issued; good return becomes Available; damaged return becomes Maintenance and history is retained.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T159, T058

BR3 - Concurrent approvals

TC-BR3-001 | Race competing requests for the last unit

Requirement ID: BR3
Module: Concurrent approvals
Test scenario: Race competing requests for the last unit

Preconditions: Isolated file-backed SQLite test database; separate worker connections and synchronized competing actions; no working DB access.

Test data: One Available unit; two Pending requests; isolated file-backed SQLite test DB

Test steps
1. Start concurrent approvals on separate connections.
2. Join both workers.
3. Count successful approvals, active allocations and unit state.

Expected result: Exactly one approval succeeds and one active reservation exists. SQLite writer contention is documented.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T160

TC-BR3-002 | Prevent duplicate decisions, issues and returns under concurrency

Requirement ID: BR3
Module: Concurrent approvals
Test scenario: Prevent duplicate decisions, issues and returns under concurrency

Preconditions: Isolated file-backed SQLite test database; separate worker connections and synchronized competing actions; no working DB access.

Test data: Same request/loan/unit targeted by two workers

Test steps
1. Race duplicate approval.
2. Race duplicate release and duplicate per-unit return.
3. Inspect unique outcomes and history.

Expected result: No duplicate reservation, loan or return is committed.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T161, T162, T163

BR4 - All-row validation

TC-BR4-001 | Validate all rows before any request write

Requirement ID: BR4
Module: All-row validation
Test scenario: Validate all rows before any request write

Preconditions: Isolated request fixtures with valid first-row data and invalid variants; baseline row counts known.

Test data: First valid row followed by duplicate/nonexistent/invalid-quantity row

Test steps
1. Submit invalid multirow data through service and HTTP.
2. Inspect request, item and allocation tables.
3. Compare counts with preconditions.

Expected result: No partial request remains after any invalid row.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T123, T132, T164

BR5 - Quantity/duration and revision

TC-BR5-001 | Reject expired release until valid recorded Admin revision

Requirement ID: BR5
Module: Quantity/duration and revision
Test scenario: Reject expired release until valid recorded Admin revision

Preconditions: Isolated time-aware agreement fixtures and policy; active Admin for revisions; expected prior date supplied where required.

Test data: Expired Approved request within original-submission policy window

Test steps
1. Attempt issue of expired approval.
2. Revise with valid future date and mandatory reason.
3. Issue and compare final loan date/audit.

Expected result: Expired issue is refused; valid Admin revision records old/new date/actor/time/reason and final date is copied into Loan.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T165, T166, T167

TC-BR5-002 | Reject invalid or late due-date revisions

Requirement ID: BR5
Module: Quantity/duration and revision
Test scenario: Reject invalid or late due-date revisions

Preconditions: Isolated time-aware agreement fixtures and policy; active Admin for revisions; expected prior date supplied where required.

Test data: Blank reason, out-of-window date, stale date, issued/completed request

Test steps
1. Try each revision against the current policy and original submission.
2. Inspect the unchanged agreement and audit.
3. Attempt a duplicate stale POST.

Expected result: Invalid/stale/post-issue revisions cannot change recorded terms or duplicate audit entries.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T168, T169, T170, T171

TC-BR5-003 | Serialize due revision with cancellation and issuance

Requirement ID: BR5
Module: Quantity/duration and revision
Test scenario: Serialize due revision with cancellation and issuance

Preconditions: Isolated time-aware agreement fixtures and policy; active Admin for revisions; expected prior date supplied where required.

Test data: Approved request concurrently revised and issued/cancelled

Test steps
1. Run isolated competing revision/issue operations.
2. Run revision/cancellation operations.
3. Inspect final agreement, loan and audit.

Expected result: Revision cannot mutate an issued/cancelled agreement or leave conflicting dates.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T172, T173

BR6 - Calendar-day overdue

TC-BR6-001 | Calculate overdue days at Manila calendar boundaries

Requirement ID: BR6
Module: Calendar-day overdue
Test scenario: Calculate overdue days at Manila calendar boundaries

Preconditions: Time-controlled Asia/Manila test dates and known outstanding/returned loan rows; no real-data writes.

Test data: Before/on due day, one second after midnight and viewer in another timezone

Test steps
1. Evaluate dates before/on/after the local due date.
2. Cross Manila midnight without 24 elapsed hours.
3. Compare overdue day values.

Expected result: Days are zero before/on due day and local calendar-date difference afterward, independent of viewer timezone.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T174, T175

TC-BR6-002 | Agree on overdue across UI, reports and CSV

Requirement ID: BR6
Module: Calendar-day overdue
Test scenario: Agree on overdue across UI, reports and CSV

Preconditions: Time-controlled Asia/Manila test dates and known outstanding/returned loan rows; no real-data writes.

Test data: Outstanding partial loan, final return, stale completion and zero loan units

Test steps
1. Read all surfaces and CSV after midnight.
2. Partially/finally return units.
3. Repeat with zero outstanding/stale completion/empty loan fixtures.

Expected result: All surfaces agree; loans with no outstanding units are never overdue.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T176, T055, T056, T177

TC-BR6-003 | Keep overdue handling free of financial features

Requirement ID: BR6
Module: Calendar-day overdue
Test scenario: Keep overdue handling free of financial features

Preconditions: Time-controlled Asia/Manila test dates and known outstanding/returned loan rows; no real-data writes.

Test data: Actual installed domain-model registry and routes

Test steps
1. Inspect overdue model/service/report fields.
2. Inspect domain models and application routes.
3. Check for penalty/payment/fine feature modules.

Expected result: Overdue monitoring computes status/days without introducing monetary features.

Actual result: Artifact/check verification (no_finance): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-no_finance

RA - Shared access control

TC-RA-001 | Enforce two database roles and explicit authorized promotion

Requirement ID: RA.1
Module: Shared access control
Test scenario: Enforce two database roles and explicit authorized promotion

Preconditions: Isolated users with known campus role/current activation and cross-owner records; forged privilege flags are fixture data.

Test data: Third-role direct writes and Basic User with staff/superuser flag

Test steps
1. Attempt invalid role create/update.
2. Try privileged account endpoints as Basic User.
3. Perform an authorized audited promotion.

Expected result: Only ADMIN/BASIC_USER roles are accepted; Django staff flags do not bypass campus authorization.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T178, T179, T180

TC-RA-002 | Recheck ownership/role and current active state on server

Requirement ID: RA.2, RA.3
Module: Shared access control
Test scenario: Recheck ownership/role and current active state on server

Preconditions: Isolated users with known campus role/current activation and cross-owner records; forged privilege flags are fixture data.

Test data: Stale actor, changed role and other-user records

Test steps
1. Change role/activation after sign-in.
2. Attempt protected requests and mutations.
3. Inspect records for unauthorized changes.

Expected result: Current database actor and owner rules apply to every protected action.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T181, T006, T144

UI - Interface usability

TC-UI-001 | Render empty states, text statuses and validated errors

Requirement ID: UI.1
Module: Interface usability
Test scenario: Render empty states, text statuses and validated errors

Preconditions: Copied local application and fictional accounts; explicit empty/populated states. Manual assistive checks require suitable tools/participants.

Test data: Empty and populated pages; invalid form; confirmation missing

Test steps
1. Open empty lists/dashboard.
2. Submit invalid fields and missing confirmation.
3. Read visible status labels and errors.

Expected result: Empty states, errors and textual statuses are visible; required confirmations protect mutations.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T182, T183, T184, T185

TC-UI-002 | Inspect actual desktop role navigation and primary layouts

Requirement ID: UI.2
Module: Interface usability
Test scenario: Inspect actual desktop role navigation and primary layouts

Preconditions: Copied local application and fictional accounts; explicit empty/populated states. Manual assistive checks require suitable tools/participants.

Test data: Fictional Admin/Basic User at 1440 by 1100 viewport

Test steps
1. Run the copied application.
2. Sign in by each role.
3. Open the 26 documented screenshot views and inspect layout.

Expected result: Actual role pages have consistent navigation and readable desktop content.

Actual result: Artifact/check verification (screenshots): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-screenshots

TC-UI-003 | Exercise phone/tablet and assistive-technology behavior

Requirement ID: UI.2
Module: Interface usability
Test scenario: Exercise phone/tablet and assistive-technology behavior

Preconditions: Copied local application and fictional accounts; explicit empty/populated states. Manual assistive checks require suitable tools/participants.

Test data: Phone/tablet browser sizes, keyboard-only and screen reader

Test steps
1. Navigate catalog, request, tables and dialogs at small widths.
2. Operate all actions by keyboard and a screen reader.
3. Record focus, scroll and error announcements.

Expected result: Responsive pages remain usable and dialogs/errors are accessible.

Actual result: This package run captured desktop views only. A complete mobile/tablet and assistive-technology session has not been executed; CSS or automated template assertions alone do not establish this result.

Status: NOT EXECUTED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

DB - Database representation/integrity

TC-DB-001 | Reconcile conceptual entities to actual migrated schema

Requirement ID: DB.1, DB.2, DB.3, DB.4, DB.5, DB.6, DB.7, DB.8
Module: Database representation/integrity
Test scenario: Reconcile conceptual entities to actual migrated schema

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Fresh migrated isolated database and Django model metadata

Test steps
1. Apply all included migrations to the isolated database.
2. Inspect actual tables, fields and relationships.
3. Compare the schema/data dictionary with twelve domain models.

Expected result: Required logical entities are represented by actual fields/relations; User embeds profile and LoanUnit embeds per-unit return data.

Actual result: Artifact/check verification (schema): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-schema

TC-DB-002 | Enforce asset uniqueness and permitted physical conditions

Requirement ID: DB.3
Module: Database representation/integrity
Test scenario: Enforce asset uniqueness and permitted physical conditions

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Duplicate normalized asset code and invalid condition

Test steps
1. Attempt duplicate/case/space asset variants.
2. Try unsupported condition via form/service/database.
3. Inspect original unit and failed writes.

Expected result: Asset uniqueness and known condition choices are enforced without status conflation.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T021, T024

TC-DB-003 | Inspect request-item and active allocation database constraints

Requirement ID: DB.4, DB.5
Module: Database representation/integrity
Test scenario: Inspect request-item and active allocation database constraints

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Migrated constraint definitions and field metadata

Test steps
1. Inspect BorrowingRequestItem constraints.
2. Inspect Allocation conditional uniqueness and item/unit relation.
3. Compare migrated definitions with the data dictionary.

Expected result: Unique request-item/active-unit and positive-quantity constraints are declared in the migrated schema.

Actual result: Artifact/check verification (schema): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-schema

TC-DB-004 | Enforce one loan/request and one outstanding issue/unit

Requirement ID: DB.6
Module: Database representation/integrity
Test scenario: Enforce one loan/request and one outstanding issue/unit

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Duplicate loan/allocation issue; related inventory history

Test steps
1. Attempt duplicate active issue/release.
2. Try deleting referenced history through inventory relations.
3. Inspect loan-unit and request relations.

Expected result: Unique/PROTECT relationships retain borrowing and per-unit return history.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T186, T187, T046

TC-DB-005 | Constrain active maintenance and singleton policy

Requirement ID: DB.7, DB.8
Module: Database representation/integrity
Test scenario: Constrain active maintenance and singleton policy

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Duplicate active repair record; unresolved closure; invalid/multiple settings

Test steps
1. Attempt invalid model writes.
2. Inspect database errors.
3. Check the valid original rows remain unchanged.

Expected result: Maintenance and settings constraints reject inconsistent records.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T188, T189, T190

TC-DB-006 | Preserve legacy data during additive migrations

Requirement ID: DB.2, DB.3
Module: Database representation/integrity
Test scenario: Preserve legacy data during additive migrations

Preconditions: Fresh migrated isolated schema and actual included Django models; invalid-write fixtures are transaction-contained.

Test data: Pre-upgrade rows lacking condition/acquisition metadata

Test steps
1. Migrate the isolated historical fixture forward.
2. Inspect identities, nullable values and related records.
3. Validate new metadata entry afterward.

Expected result: No fabricated acquisition dates or destructive relationship rewrite is introduced.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T191

TECH - Framework and security

TC-TECH-001 | Verify Django structure and custom Admin architecture

Requirement ID: TECH.1, TECH.2
Module: Framework and security
Test scenario: Verify Django structure and custom Admin architecture

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: Copied source tree, routes, role bases and installed app settings

Test steps
1. Inspect models/forms/views/templates/URLs and named routes.
2. Resolve /admin/ and inspect Admin source.
3. Run Django checks and migration consistency.

Expected result: The application uses Django MTV/ORM and custom role portals; generated Django Admin is not exposed.

Actual result: Artifact/check verification (framework): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-framework

TC-TECH-002 | Render custom Admin navigation without generated Admin

Requirement ID: TECH.2
Module: Framework and security
Test scenario: Render custom Admin navigation without generated Admin

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: Authorized Admin and all sidebar destinations

Test steps
1. GET each custom Admin destination.
2. Inspect role base and navigation.
3. Check for writes during read-only page loads.

Expected result: Custom pages render with shared navigation and GET does not mutate business records.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T192, T193

TC-TECH-003 | Preserve related filtering, pagination and computed reporting

Requirement ID: TECH.3
Module: Framework and security
Test scenario: Preserve related filtering, pagination and computed reporting

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: Combined inventory filters and multipage report rows

Test steps
1. Apply category/item/status/search together.
2. Navigate result pages and reset.
3. Compare exported related-model totals with database rows.

Expected result: Filters/pagination retain state and computed reports use actual related data.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T026, T027, T076

TC-TECH-004 | Protect every state-changing workflow against missing CSRF

Requirement ID: TECH.4
Module: Framework and security
Test scenario: Protect every state-changing workflow against missing CSRF

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: Missing/invalid CSRF tokens on account, inventory, borrowing, loan and maintenance actions

Test steps
1. POST each mutation without valid CSRF.
2. Try GET mutation routes.
3. Inspect response and unchanged database.

Expected result: CSRF and POST-only restrictions prevent unauthorized state changes.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T194, T195, T196, T197, T198

TC-TECH-005 | Escape untrusted content and keep protected notes private

Requirement ID: TECH.4
Module: Framework and security
Test scenario: Escape untrusted content and keep protected notes private

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: HTML-like audit/equipment text and formula-like CSV data

Test steps
1. Render untrusted strings in detail/audit pages.
2. Export risky CSV values.
3. Inspect Basic User pages for internal data.

Expected result: Templates escape HTML, CSV handles spreadsheet formula values, and private notes are not disclosed.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T199, T117, T077, T200

TC-TECH-006 | Install pinned dependencies and run the submission copy

Requirement ID: TECH.5
Module: Framework and security
Test scenario: Install pinned dependencies and run the submission copy

Preconditions: Separate copied source, pinned environment and isolated fixture configuration; no production deployment.

Test data: New isolated Python environment and copied app, external fictional DB

Test steps
1. Install the pinned requirements.
2. Run pip check and Django checks.
3. Migrate/seed/verify and serve the copied application on loopback.

Expected result: Dependencies resolve and the copied application runs independently of the working database.

Actual result: Artifact/check verification (dependencies): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-dependencies

DEMO - Required fictional demonstration

TC-DEMO-001 | Recreate and reconcile required fictional demonstration records

Requirement ID: DEMO.1, DEMO.2
Module: Required fictional demonstration
Test scenario: Recreate and reconcile required fictional demonstration records

Preconditions: Clean isolated database with explicit seed confirmation; private locally generated fictional credentials.

Test data: Clean isolated database; confirmed seed command

Test steps
1. Run seed_project_demo with isolation confirmation.
2. Check every minimum count and required scenario.
3. Repeat the read-only verifier and idempotence/safety tests.

Expected result: Fictional minimum counts and historical scenarios reconcile; unsafe working/nonempty databases are refused.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T201, T202, T203, T204

TC-DEMO-002 | Keep historical fixtures separate from normal date validation

Requirement ID: DEMO.2, BR5
Module: Required fictional demonstration
Test scenario: Keep historical fixtures separate from normal date validation

Preconditions: Clean isolated database with explicit seed confirmation; private locally generated fictional credentials.

Test data: Historical demo records and ordinary new-request service

Test steps
1. Inspect confirmed isolated seeding.
2. Submit normal past/out-of-window due dates.
3. Inspect rejection and existing fixture preservation.

Expected result: Historical fixtures do not weaken ordinary future/date policy validation.

Actual result: Automated assertions for the cited methods passed in the isolated Django test run. The numbered procedure is a reproducible scenario; it is not a claim that a human executed every UI step.

Status: PASS
Execution mode: Automated service/HTTP assertions
Evidence: T133, T201

SUB - Submission delivery

TC-SUB-001 | Validate ZIP structure, content and excluded private artifacts

Requirement ID: SUB.1
Module: Submission delivery
Test scenario: Validate ZIP structure, content and excluded private artifacts

Preconditions: Final copied package and ZIP; source snapshot baseline; private scan values remain outside the artifact.

Test data: Final ZIP, required source paths, four PDFs and 26 actual PNGs

Test steps
1. Open and test the final ZIP.
2. Check required paths, member hashes and file exclusions.
3. Scan for private configuration/credentials and compare copied source hashes.

Expected result: The ZIP has the required project root, source/tests/migrations/setup/docs and no private DB/environment/cache/venv/Git artifacts.

Actual result: Artifact/check verification (package): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-package

TC-SUB-002 | Deliver private demonstration credentials to instructor

Requirement ID: SUB.2
Module: Submission delivery
Test scenario: Deliver private demonstration credentials to instructor

Preconditions: Final copied package and ZIP; source snapshot baseline; private scan values remain outside the artifact.

Test data: One Admin and two Basic User accounts in recipient isolated demo

Test steps
1. Generate private local demonstration credentials.
2. Use the instructor-approved private channel.
3. Record authorized receipt without publishing passwords.

Expected result: Instructor receives working credentials privately; the public ZIP contains no secrets.

Actual result: The instructor channel and recipient have not been supplied. The included demo helper generates credentials locally; delivery itself has not occurred.

Status: BLOCKED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

EVID - Academic and professional evidence

TC-EVID-001 | Validate acceptance and real stakeholder feedback evidence

Requirement ID: EVID.1
Module: Academic and professional evidence
Test scenario: Validate acceptance and real stakeholder feedback evidence

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Actual dated stakeholder criteria, results and issue resolution records

Test steps
1. Have authorized users exercise agreed criteria.
2. Record observations and accepted outcomes.
3. Link feedback to resolved actions and maintenance guidance.

Expected result: Actual acceptance/feedback evidence supports user-needs evaluation.

Actual result: No verified stakeholder interview, acceptance sign-off or feedback-resolution record was provided. Documentation includes a requirements-based analysis and a proposed evaluation procedure, not invented participant results.

Status: BLOCKED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

TC-EVID-002 | Validate rendered technical documentation and diagrams

Requirement ID: EVID.2
Module: Academic and professional evidence
Test scenario: Validate rendered technical documentation and diagrams

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Four final PDFs, actual schema, source and execution ledger

Test steps
1. Open/render every generated PDF.
2. Inspect TOC/page numbers/diagrams/tables.
3. Compare model mapping, code references and test results with current artifacts.

Expected result: Readable final PDFs contain accurate rendered diagrams and evidence-qualified technical/ethical discussion.

Actual result: Artifact/check verification (pdfs): verified against this build; see evidence register.

Status: PASS
Execution mode: Build/artifact inspection
Evidence: Q-pdfs

TC-EVID-003 | Confirm institutional/legal/accessibility review

Requirement ID: EVID.3
Module: Academic and professional evidence
Test scenario: Confirm institutional/legal/accessibility review

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Instructor/BulSU policy and authorized reviewers

Test steps
1. Review privacy/IP/asset attribution and accessibility risks.
2. Confirm applicable policy with the instructor.
3. Record decisions and accountable owners.

Expected result: Real policy review substantiates safeguards; no unsupported legal or accessibility certification is claimed.

Actual result: No institutional policy ruling or formal accessibility/legal review was provided. The documentation supplies risk analysis and review actions, not a compliance certification.

Status: BLOCKED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

TC-EVID-004 | Verify actual group membership and contribution evidence

Requirement ID: EVID.4
Module: Academic and professional evidence
Test scenario: Verify actual group membership and contribution evidence

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Official section, five-to-six verified members and dated contribution records

Test steps
1. Confirm roster/student identifiers privately.
2. Inspect commits/contributions and leadership rotation.
3. Attach peer feedback and performance improvements.

Expected result: Each member has attributable genuine work and collaboration evidence.

Actual result: Section, verified roster and individual contribution/leadership/peer records were not supplied. They must be completed by the group; no names or participation are fabricated.

Status: BLOCKED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

TC-EVID-005 | Conduct the timed demonstration and Q&A

Requirement ID: EVID.5
Module: Academic and professional evidence
Test scenario: Conduct the timed demonstration and Q&A

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Final local demo, audience, actual assigned speakers and feedback form

Test steps
1. Run the proposed twelve-minute agenda.
2. Allow three minutes for questions.
3. Record speaker segments, timing, questions and follow-up actions.

Expected result: A real demonstration establishes communication and feedback outcomes.

Actual result: A timed group presentation, Q&A and audience feedback collection have not taken place during this preparation. The guide provides an agenda only.

Status: NOT EXECUTED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

TC-EVID-006 | Execute disclosure and demonstrate individual understanding

Requirement ID: EVID.6
Module: Academic and professional evidence
Test scenario: Execute disclosure and demonstrate individual understanding

Preconditions: Official project evidence requirements, actual proponents/instructor/authorized stakeholders and authentic records as indicated.

Test data: Draft disclosure, instructor AI policy and proponents

Test steps
1. Verify and amend the assistance/asset declaration.
2. Obtain real signatures and instructor policy confirmation.
3. Ask each member to explain relevant implementation decisions.

Expected result: The signed accurate declaration and individual explanation comply with applicable course policy.

Actual result: The AI agreement is an unsigned draft. Real signatures, individual explanations, instructor policy approval and individual rubric evidence cannot be generated by this task.

Status: BLOCKED
Execution mode: Manual/external evidence
Evidence: No passing execution record; see stated blocker/limitation.

4. Exact automated evidence register

Every T-number below resolves to an actually executed PASS result in run IT401W-G1-20261025. The permanent tests are included in the source ZIP. References use repository-relative paths and verified method start lines from the copied source. The source may be wrapped typographically in this table; the method identifier itself is unchanged. Multiple scenario references to one T-number do not represent multiple executions.

T001 | PASS

apps/accounts/tests.py:376

AccountManagementTests.test_search_username_full_name_email_and_combined_filters

T002 | PASS

apps/accounts/tests.py:397

AccountManagementTests.test_activate_and_deactivate_post_are_audited

T003 | PASS

apps/accounts/tests.py:468

AccountManagementTests.test_deactivation_preserves_borrowing_and_return_history

T004 | PASS

apps/accounts/tests.py:456

AccountManagementTests.test_history_scopes_borrowing_loans_and_returns_to_target_user

T005 | PASS

apps/accounts/tests.py:201

AuthenticationTests.test_inactive_users_cannot_login

T006 | PASS

apps/accounts/tests.py:564

AccountServiceSecurityTests.test_deactivated_existing_sessions_lose_authenticated_access

T007 | PASS

apps/accounts/tests.py:578

AccountServiceSecurityTests.test_stale_borrower_cannot_submit_after_deactivation

T008 | PASS

apps/accounts/tests.py:164

AuthenticationTests.test_public_registration_cannot_promote_or_assign_permissions

T009 | PASS

apps/inventory/tests.py:57

InventoryServiceTests.test_category_create_edit_and_trim

T010 | PASS

apps/inventory/tests.py:564

InventoryPageTests.test_admin_category_create_edit_duplicate_and_delete_routes

T011 | PASS

apps/inventory/test_a2_requirements.py:23

CategoryArchiveTests.test_archive_restore_keep_relationships_stock_and_existing_borrowing

T012 | PASS

apps/inventory/test_a2_requirements.py:38

CategoryArchiveTests.test_archived_category_cannot_be_assigned_to_new_or_different_equipment

T013 | PASS

apps/inventory/test_a2_requirements.py:54

CategoryArchiveTests.test_edit_existing_equipment_can_keep_its_archived_category_or_choose_active_one

T014 | PASS

apps/inventory/test_a2_requirements.py:65

CategoryArchiveTests.test_names_remain_unique_and_edit_does_not_restore_archived_category

T015 | PASS

apps/inventory/tests.py:71

InventoryServiceTests.test_database_prevents_case_duplicate_category

T016 | PASS

apps/inventory/tests.py:580

InventoryPageTests.test_catalog_create_edit_and_forged_stock_state_ignored

T017 | PASS

apps/inventory/tests.py:98

InventoryServiceTests.test_catalog_create_edit_requires_existing_category

T018 | PASS

apps/inventory/tests.py:117

InventoryServiceTests.test_archive_and_restore_preserve_units_and_statuses

T019 | PASS

apps/inventory/tests.py:152

InventoryServiceTests.test_reserved_and_issued_catalogs_cannot_be_archived

T020 | PASS

apps/inventory/tests.py:169

InventoryServiceTests.test_unit_create_normalization_and_unique_identifier

T021 | PASS

apps/inventory/tests.py:177

InventoryServiceTests.test_database_enforces_unique_identifier_case_and_trim

T022 | PASS

apps/inventory/tests.py:181

InventoryServiceTests.test_invalid_unit_identifier

T023 | PASS

apps/inventory/test_a2_requirements.py:116

PhysicalUnitMetadataTests.test_known_conditions_and_dates_are_stored_without_changing_status

T024 | PASS

apps/inventory/test_a2_requirements.py:137

PhysicalUnitMetadataTests.test_invalid_condition_is_rejected_by_form_service_and_database

T025 | PASS

apps/inventory/test_a2_requirements.py:147

PhysicalUnitMetadataTests.test_acquisition_date_validation_rejects_future_and_invalid_dates

T026 | PASS

apps/inventory/test_a2_requirements.py:232

PhysicalUnitCategoryFilterTests.test_category_item_status_and_search_work_alone_and_together

T027 | PASS

apps/inventory/test_a2_requirements.py:257

PhysicalUnitCategoryFilterTests.test_pagination_retains_combined_filters_and_reset_clears_them

T028 | PASS

apps/inventory/tests.py:88

InventoryServiceTests.test_referenced_category_cannot_be_deleted_even_when_equipment_archived

T029 | PASS

apps/inventory/tests.py:574

InventoryPageTests.test_referenced_category_deletion_denied_in_backend

T030 | PASS

apps/borrowing/tests.py:494

BorrowingPageTests.test_admin_queue_defaults_pending_and_all_search_status_filters_work

T031 | PASS

apps/borrowing/tests.py:610

BorrowingPageTests.test_request_pagination_keeps_filters_and_handles_invalid_pages

T032 | PASS

apps/borrowing/test_compliance.py:24

RequestComplianceTests.test_date_boundaries_combined_filters_and_pagination_use_manila

T033 | PASS

apps/borrowing/test_compliance.py:46

RequestComplianceTests.test_invalid_date_or_reversed_range_displays_errors

T034 | PASS

apps/borrowing/test_compliance.py:53

RequestComplianceTests.test_review_counts_exclude_each_unavailable_status_and_refresh

T035 | PASS

apps/borrowing/tests.py:509

BorrowingPageTests.test_get_review_pages_never_change_state_or_reserve

T036 | PASS

apps/borrowing/test_compliance.py:71

RequestComplianceTests.test_approval_requires_reason_in_form_and_service

T037 | PASS

apps/borrowing/test_compliance.py:85

RequestComplianceTests.test_approval_reason_persisted_escaped_and_audited_with_actor_time

T038 | PASS

apps/borrowing/tests.py:200

BorrowingServiceTests.test_rejection_records_reason_date_and_admin

T039 | PASS

apps/borrowing/tests.py:542

BorrowingPageTests.test_reject_requires_reason_and_displays_escaped_reason_to_borrower

T040 | PASS

apps/borrowing/tests.py:141

BorrowingServiceTests.test_successful_automatic_allocation_reserves_exact_quantities

T041 | PASS

apps/loans/tests.py:334

FoundationTests.test_approval_reserves_exact_units_and_records_actor

T042 | PASS

apps/loans/tests.py:69

LoanServiceTests.test_successful_release_and_snapshots

T043 | PASS

apps/loans/tests.py:585

LoanPageTests.test_printable_slip_contains_units_and_handover_details

T044 | PASS

apps/loans/tests.py:431

LoanPageTests.test_basic_user_cannot_release_or_return_or_print_admin_slip

T045 | PASS

apps/loans/tests.py:103

LoanServiceTests.test_non_approved_requests_cannot_be_released

T046 | PASS

apps/loans/tests.py:96

LoanServiceTests.test_duplicate_release

T047 | PASS

apps/loans/tests.py:125

LoanServiceTests.test_release_rejects_incomplete_allocation

T048 | PASS

apps/loans/tests.py:141

LoanServiceTests.test_release_rejects_wrong_equipment_allocation

T049 | PASS

apps/loans/tests.py:150

LoanServiceTests.test_release_rolls_back_when_one_unit_is_unavailable

T050 | PASS

apps/loans/tests.py:174

LoanServiceTests.test_partial_return

T051 | PASS

apps/loans/tests.py:203

LoanServiceTests.test_last_partial_return_completes_loan

T052 | PASS

apps/loans/tests.py:191

LoanServiceTests.test_full_return_completes_loan_and_request

T053 | PASS

apps/loans/tests.py:221

LoanServiceTests.test_duplicate_return_is_rejected_without_overwriting_record

T054 | PASS

apps/loans/tests.py:254

LoanServiceTests.test_units_cannot_be_returned_against_another_loan

T055 | PASS

apps/loans/test_overdue.py:82

CalendarOverdueTests.test_partial_and_final_returns_update_overdue_classification_and_days

T056 | PASS

apps/loans/test_overdue.py:99

CalendarOverdueTests.test_stale_completion_never_makes_zero_outstanding_loan_overdue_anywhere

T057 | PASS

apps/loans/tests.py:211

LoanServiceTests.test_damaged_return_routes_unit_to_maintenance

T058 | PASS

apps/maintenance/tests.py:260

MaintenanceIntegrationTests.test_damaged_return_creates_linked_record_atomically

T059 | PASS

apps/maintenance/tests.py:65

MaintenanceServiceTests.test_create_changes_unit_status_and_records_admin_and_date

T060 | PASS

apps/maintenance/test_quick_actions.py:32

MaintenanceQuickActionTests.test_start_preserves_reporting_admin_notes_and_records_start_actor_and_time

T061 | PASS

apps/maintenance/test_quick_actions.py:122

MaintenanceQuickActionTests.test_completion_repaired_restores_available_and_records_actor_date_and_work

T062 | PASS

apps/maintenance/test_quick_actions.py:140

MaintenanceQuickActionTests.test_completion_retired_remains_unavailable_and_closed_history_cannot_restart

T063 | PASS

apps/maintenance/test_quick_actions.py:106

MaintenanceQuickActionTests.test_completion_modal_requires_progress_notes_confirmation_and_valid_outcome

T064 | PASS

apps/maintenance/tests.py:133

MaintenanceServiceTests.test_progress_cannot_reverse_or_complete

T065 | PASS

apps/maintenance/test_quick_actions.py:197

MaintenanceBorrowingGuardTests.test_reserved_and_issued_units_reject_new_maintenance_in_form_and_service

T066 | PASS

apps/maintenance/tests.py:347

MaintenanceIntegrationTests.test_active_allocation_blocks_manual_creation_even_with_wrong_status

T067 | PASS

apps/inventory/tests.py:213

InventoryServiceTests.test_all_status_counts_are_calculated_from_physical_units

T068 | PASS

apps/inventory/tests.py:264

InventoryServiceTests.test_loan_release_good_and_damaged_returns_update_counts

T069 | PASS

apps/dashboard/tests.py:133

StatisticsAndChartsTests.test_kpis_count_physical_units_and_loans

T070 | PASS

apps/dashboard/test_compliance.py:24

DashboardComplianceTests.test_required_kpis_and_charts_render_and_pending_changes

T071 | PASS

apps/dashboard/test_compliance.py:35

DashboardComplianceTests.test_monthly_chart_counts_loans_once_not_requests_or_physical_units

T072 | PASS

apps/dashboard/test_compliance.py:46

DashboardComplianceTests.test_category_usage_counts_each_issuance_and_preserves_returned_history

T073 | PASS

apps/dashboard/tests.py:279

ReportsTests.test_loan_report_issue_date_filter_matches_export

T074 | PASS

apps/dashboard/tests.py:290

ReportsTests.test_overdue_report_filters_due_dates_not_issue_dates

T075 | PASS

apps/dashboard/tests.py:382

ReportsTests.test_local_date_boundaries_match_display_and_csv

T076 | PASS

apps/dashboard/tests.py:359

ReportsTests.test_report_pagination_and_export_includes_all_filtered_rows

T077 | PASS

apps/dashboard/tests.py:342

ReportsTests.test_csv_escapes_formula_like_user_values_and_quotes_commas

T078 | PASS

apps/dashboard/tests.py:328

ReportsTests.test_csv_headers_format_and_no_sensitive_notes

T079 | PASS

apps/dashboard/tests.py:303

ReportsTests.test_partial_return_report_contains_each_unit_and_correct_status

T080 | PASS

apps/dashboard/tests.py:353

ReportsTests.test_report_uses_issue_snapshots_after_inventory_rename

T081 | PASS

apps/dashboard/tests.py:560

AuditIntegrationTests.test_approval_rejection_and_cancellation_are_audited_once

T082 | PASS

apps/dashboard/tests.py:572

AuditIntegrationTests.test_release_return_and_automatic_maintenance_have_distinct_audit_events

T083 | PASS

apps/dashboard/tests.py:533

AuditIntegrationTests.test_inventory_create_update_archive_and_restore_are_audited

T084 | PASS

apps/dashboard/tests.py:595

AuditIntegrationTests.test_failed_operations_produce_no_audit_or_partial_changes

T085 | PASS

apps/dashboard/tests.py:603

AuditIntegrationTests.test_audit_failure_rolls_back_inventory_creation

T086 | PASS

apps/dashboard/tests.py:644

AuditViewTests.test_search_by_actor_action_entity_and_record_id

T087 | PASS

apps/dashboard/tests.py:653

AuditViewTests.test_date_filter_and_newest_first_order

T088 | PASS

apps/dashboard/tests.py:88

DashboardAccessTests.test_basic_cannot_access_admin_pages_or_export_even_as_superuser

T089 | PASS

apps/dashboard/tests.py:396

SettingsTests.test_default_limits_are_three_three_seven_without_creating_a_row

T090 | PASS

apps/dashboard/tests.py:421

SettingsTests.test_numeric_type_and_range_validation

T091 | PASS

apps/dashboard/tests.py:471

SettingsTests.test_audit_failure_rolls_back_policy_update

T092 | PASS

apps/dashboard/tests.py:476

SettingsTests.test_new_submissions_obey_saved_quantities_duration_and_distinct_limits

T093 | PASS

apps/dashboard/tests.py:495

SettingsTests.test_existing_approved_request_can_release_after_limits_are_lowered

T094 | PASS

apps/dashboard/tests.py:504

SettingsTests.test_settings_changes_do_not_modify_issued_loans_or_returns

T095 | PASS

apps/accounts/tests.py:147

AuthenticationTests.test_registration_creates_basic_user_and_authenticated_session

T096 | PASS

apps/accounts/tests.py:157

AuthenticationTests.test_registration_uses_django_secure_password_hashing

T097 | PASS

apps/accounts/tests.py:171

AuthenticationTests.test_duplicate_username_is_rejected

T098 | PASS

apps/accounts/tests.py:176

AuthenticationTests.test_required_fields_username_email_and_contact_validation

T099 | PASS

apps/accounts/tests.py:183

AuthenticationTests.test_password_strength_similarity_and_confirmation_validation

T100 | PASS

apps/accounts/tests.py:195

AuthenticationTests.test_invalid_credentials_do_not_authenticate

T101 | PASS

apps/accounts/tests.py:189

AuthenticationTests.test_basic_and_admin_login_redirects

T102 | PASS

apps/accounts/tests.py:207

AuthenticationTests.test_unsupported_legacy_roles_cannot_login

T103 | PASS

apps/accounts/tests.py:223

AuthenticationTests.test_logout_is_post_only_and_flushes_session

T104 | PASS

apps/accounts/tests.py:230

AuthenticationTests.test_registration_login_and_logout_preserve_csrf

T105 | PASS

apps/accounts/tests.py:257

ProfileTests.test_profile_updates_only_allowed_fields

T106 | PASS

apps/accounts/tests.py:270

ProfileTests.test_profile_validation_errors_do_not_save_partial_values

T107 | PASS

apps/accounts/tests.py:277

ProfileTests.test_profile_has_no_other_user_id_endpoint

T108 | PASS

apps/accounts/tests.py:285

ProfileTests.test_basic_user_cannot_access_admin_profile_aliases

T109 | PASS

apps/accounts/tests.py:327

ProfileTests.test_stale_profile_cannot_reactivate_a_deactivated_account

T110 | PASS

apps/accounts/tests.py:334

ProfileTests.test_stale_profile_cannot_overwrite_a_role_change

T111 | PASS

apps/inventory/tests.py:413

InventoryPageTests.test_catalog_search_category_and_availability_filters

T112 | PASS

apps/inventory/tests.py:477

InventoryPageTests.test_combined_catalog_filters_and_pagination_preserve_query_parameters

T113 | PASS

apps/inventory/tests.py:470

InventoryPageTests.test_archived_items_hidden_and_detail_returns_404

T114 | PASS

apps/inventory/tests.py:505

InventoryPageTests.test_catalog_empty_state_when_only_archived_items_exist

T115 | PASS

apps/inventory/tests.py:361

ServiceableInventoryTests.test_public_pages_display_mixed_serviceable_totals

T116 | PASS

apps/inventory/tests.py:366

ServiceableInventoryTests.test_public_pages_display_zero_inventory

T117 | PASS

apps/inventory/tests.py:532

InventoryPageTests.test_equipment_detail_keeps_long_names_and_complete_escaped_descriptions

T118 | PASS

apps/inventory/tests.py:370

ServiceableInventoryTests.test_public_totals_follow_reservation_issue_return_and_maintenance_transitions

T119 | PASS

apps/inventory/tests.py:459

InventoryPageTests.test_catalog_and_details_never_expose_identifiers_internal_counts_or_admin_controls

T120 | PASS

apps/maintenance/tests.py:531

MaintenanceViewTests.test_public_catalog_never_exposes_internal_notes_or_repair_actions

T121 | PASS

apps/borrowing/tests.py:60

BorrowingServiceTests.test_submit_one_two_or_three_distinct_catalog_items

T122 | PASS

apps/borrowing/tests.py:402

BorrowingPageTests.test_http_one_two_and_three_item_submissions

T123 | PASS

apps/borrowing/tests.py:82

BorrowingServiceTests.test_duplicate_and_nonexistent_catalog_items

T124 | PASS

apps/borrowing/tests.py:432

BorrowingPageTests.test_http_oversized_or_missing_management_form_and_blank_request

T125 | PASS

apps/borrowing/tests.py:877

TwoStepBorrowingPageTests.test_forged_step_state_does_not_bypass_selection_validation

T126 | PASS

apps/borrowing/tests.py:368

BorrowingHardItemLimitTests.test_service_caps_manipulated_django_setting

T127 | PASS

apps/borrowing/tests.py:375

BorrowingHardItemLimitTests.test_service_caps_legacy_database_policy

T128 | PASS

apps/borrowing/tests.py:372

BorrowingHardItemLimitTests.test_frontend_and_forged_post_cap_manipulated_django_setting

T129 | PASS

apps/dashboard/tests.py:443

SettingsTests.test_admin_cannot_raise_item_limit_or_change_other_values_on_invalid_post

T130 | PASS

apps/borrowing/tests.py:76

BorrowingServiceTests.test_quantity_limit_and_invalid_quantities

T131 | PASS

apps/borrowing/tests.py:114

BorrowingServiceTests.test_purpose_validation

T132 | PASS

apps/borrowing/tests.py:417

BorrowingPageTests.test_http_quantity_item_duplicate_and_duration_limits

T133 | PASS

apps/borrowing/tests.py:88

BorrowingServiceTests.test_duration_limit_past_and_naive_dates

T134 | PASS

apps/borrowing/tests.py:94

BorrowingServiceTests.test_seven_calendar_day_boundary_in_local_timezone

T135 | PASS

apps/borrowing/tests.py:105

BorrowingServiceTests.test_configurable_limits_enforced_by_service

T136 | PASS

apps/borrowing/tests.py:845

TwoStepBorrowingPageTests.test_unavailable_catalog_item_is_disabled_and_cannot_be_forged

T137 | PASS

apps/borrowing/tests.py:449

BorrowingPageTests.test_submission_handles_insufficient_quantity_without_creating_request

T138 | PASS

apps/borrowing/tests.py:797

DynamicEquipmentFormTests.test_dynamic_one_two_three_row_submissions_stay_pending_without_reservations

T139 | PASS

apps/borrowing/tests.py:439

BorrowingPageTests.test_forged_status_borrower_and_availability_are_ignored

T140 | PASS

apps/borrowing/tests.py:895

EquipmentPreselectionFlowTests.test_catalog_link_preselects_item_and_places_selected_details_before_catalog

T141 | PASS

apps/borrowing/tests.py:919

EquipmentPreselectionFlowTests.test_direct_navigation_has_no_selected_item_and_shows_normal_catalog

T142 | PASS

apps/borrowing/tests.py:954

EquipmentPreselectionFlowTests.test_unavailable_zero_stock_and_archived_items_are_never_preselected

T143 | PASS

apps/borrowing/tests.py:773

DynamicEquipmentFormTests.test_validation_errors_keep_dynamic_rows_and_all_entered_values

T144 | PASS

apps/borrowing/tests.py:463

BorrowingPageTests.test_basic_user_isolation_for_list_search_details_and_cancellation

T145 | PASS

apps/loans/tests.py:439

LoanPageTests.test_basic_user_data_isolation_for_loans_and_details

T146 | PASS

apps/loans/tests.py:450

LoanPageTests.test_basic_user_history_contains_only_own_completed_loans

T147 | PASS

apps/dashboard/tests.py:237

PersonalDashboardTests.test_counts_recent_requests_and_upcoming_loans_belong_to_user

T148 | PASS

apps/borrowing/tests.py:219

BorrowingServiceTests.test_pending_cancellation_records_borrower_and_does_not_reserve

T149 | PASS

apps/borrowing/tests.py:227

BorrowingServiceTests.test_approved_cancellation_releases_reservations_and_preserves_history

T150 | PASS

apps/borrowing/tests.py:244

BorrowingServiceTests.test_basic_user_cannot_cancel_another_users_request

T151 | PASS

apps/borrowing/tests.py:567

BorrowingPageTests.test_issued_request_cannot_be_cancelled_through_any_route

T152 | PASS

apps/borrowing/tests.py:283

BorrowingServiceTests.test_cancelled_request_cannot_be_issued_and_units_can_be_reserved_again

T153 | PASS

apps/borrowing/tests.py:474

BorrowingPageTests.test_basic_users_staff_and_unknown_roles_cannot_review

T154 | PASS

apps/borrowing/test_due_revisions.py:179

DueDateRevisionTests.test_http_authorization_csrf_and_forged_actor

T155 | PASS

apps/borrowing/tests.py:151

BorrowingServiceTests.test_approval_revalidates_availability_and_rolls_back_every_item

T156 | PASS

apps/loans/tests.py:351

FoundationTests.test_approval_rolls_back_all_reservations_if_a_unit_is_unavailable

T157 | PASS

apps/borrowing/tests.py:694

ConcurrentBorrowingTests.test_cancellation_and_issuance_cannot_partially_release_the_same_reservation

T158 | PASS

apps/borrowing/tests.py:162

BorrowingServiceTests.test_approval_skips_reserved_maintenance_retired_and_issued_units

T159 | PASS

apps/borrowing/tests.py:293

BorrowingServiceTests.test_approval_to_issue_partial_return_and_full_return_integration

T160 | PASS

apps/borrowing/tests.py:680

ConcurrentBorrowingTests.test_competing_requests_cannot_reserve_the_same_units

T161 | PASS

apps/borrowing/tests.py:689

ConcurrentBorrowingTests.test_same_request_cannot_be_approved_twice

T162 | PASS

apps/loans/tests.py:738

ConcurrentLoanTests.test_two_simultaneous_releases_issue_only_one_loan

T163 | PASS

apps/loans/tests.py:744

ConcurrentLoanTests.test_two_simultaneous_returns_record_only_one_return

T164 | PASS

apps/borrowing/tests.py:762

DynamicEquipmentFormTests.test_invalid_row_actions_or_management_data_never_submit

T165 | PASS

apps/loans/tests.py:161

LoanServiceTests.test_release_rejects_expired_due_date

T166 | PASS

apps/borrowing/test_due_revisions.py:61

DueDateRevisionTests.test_expired_approval_can_be_revised_then_issued

T167 | PASS

apps/borrowing/test_due_revisions.py:44

DueDateRevisionTests.test_revision_records_complete_audit_and_issue_copies_final_date

T168 | PASS

apps/borrowing/test_due_revisions.py:80

DueDateRevisionTests.test_reason_and_due_date_validation_leave_agreement_unchanged

T169 | PASS

apps/borrowing/test_due_revisions.py:91

DueDateRevisionTests.test_revision_uses_current_policy_and_original_submission_calendar

T170 | PASS

apps/borrowing/test_due_revisions.py:103

DueDateRevisionTests.test_stale_revision_and_duplicate_post_cannot_overwrite_or_duplicate_audit

T171 | PASS

apps/borrowing/test_due_revisions.py:130

DueDateRevisionTests.test_issued_and_completed_loans_cannot_be_revised_even_with_forged_status

T172 | PASS

apps/borrowing/test_due_revisions.py:243

DueDateRevisionConcurrencyTests.test_revision_racing_issue_never_changes_date_after_issuance

T173 | PASS

apps/borrowing/test_due_revisions.py:274

DueDateRevisionConcurrencyTests.test_revision_racing_cancellation_never_changes_cancelled_agreement

T174 | PASS

apps/loans/test_overdue.py:26

CalendarOverdueTests.test_calendar_boundaries_ignore_elapsed_days_and_viewer_timezone

T175 | PASS

apps/dashboard/test_compliance.py:65

DashboardComplianceTests.test_same_local_due_date_remains_active_across_dashboard_views_reports_and_csv

T176 | PASS

apps/loans/test_overdue.py:56

CalendarOverdueTests.test_all_surfaces_and_csv_agree_one_second_after_manila_midnight

T177 | PASS

apps/loans/test_overdue.py:125

CalendarOverdueTests.test_loan_with_no_unit_records_is_not_overdue

T178 | PASS

apps/accounts/test_configuration.py:42

RoleConstraintTests.test_direct_create_and_update_cannot_introduce_third_role

T179 | PASS

apps/accounts/tests.py:363

AccountManagementTests.test_basic_superuser_cannot_access_admin_endpoints

T180 | PASS

apps/accounts/tests.py:425

AccountManagementTests.test_role_promotion_is_explicit_admin_only_and_audited

T181 | PASS

apps/accounts/tests.py:571

AccountServiceSecurityTests.test_role_change_is_enforced_in_existing_sessions

T182 | PASS

apps/borrowing/tests.py:600

BorrowingPageTests.test_empty_states_and_navigation

T183 | PASS

apps/dashboard/tests.py:167

StatisticsAndChartsTests.test_empty_database_dashboard_has_zero_cards_and_both_charts

T184 | PASS

apps/loans/tests.py:417

LoanPageTests.test_release_requires_confirmation_and_post_redirects_to_loan

T185 | PASS

apps/maintenance/tests.py:491

MaintenanceViewTests.test_completion_requires_notes_confirmation_and_progress

T186 | PASS

apps/loans/tests.py:319

LoanServiceTests.test_database_enforces_single_active_issue

T187 | PASS

apps/loans/tests.py:312

LoanServiceTests.test_history_records_cannot_be_deleted_through_related_inventory

T188 | PASS

apps/maintenance/tests.py:115

MaintenanceServiceTests.test_database_blocks_duplicate_active_records

T189 | PASS

apps/maintenance/tests.py:120

MaintenanceServiceTests.test_database_blocks_unresolved_completed_record

T190 | PASS

apps/dashboard/tests.py:429

SettingsTests.test_database_enforces_singleton_and_limit_ranges

T191 | PASS

apps/inventory/test_a2_requirements.py:201

InventoryMigrationSafetyTests.test_existing_rows_and_relationships_keep_unknown_metadata_after_upgrade

T192 | PASS

apps/dashboard/tests.py:96

DashboardAccessTests.test_admin_pages_and_sidebar_destinations_render

T193 | PASS

apps/inventory/tests.py:667

InventoryPageTests.test_get_admin_pages_render_without_modifying_data

T194 | PASS

apps/accounts/tests.py:445

AccountManagementTests.test_admin_mutations_require_csrf

T195 | PASS

apps/inventory/tests.py:703

InventoryPageTests.test_csrf_on_all_state_changing_endpoints

T196 | PASS

apps/borrowing/tests.py:590

BorrowingPageTests.test_csrf_required_on_submission_and_all_actions

T197 | PASS

apps/loans/tests.py:577

LoanPageTests.test_csrf_is_required_on_release_and_returns

T198 | PASS

apps/maintenance/tests.py:433

MaintenanceViewTests.test_csrf_is_required_on_every_mutation

T199 | PASS

apps/dashboard/tests.py:684

AuditViewTests.test_untrusted_audit_text_is_escaped_in_html

T200 | PASS

apps/dashboard/tests.py:249

PersonalDashboardTests.test_data_isolation_excludes_other_borrowers_and_internal_notes

T201 | PASS

apps/inventory/test_demo.py:33

ProjectDemoTests.test_complete_dataset_reconciles_and_is_repeatable

T202 | PASS

apps/inventory/test_demo.py:62

ProjectDemoTests.test_refuses_nonempty_or_working_database_and_missing_confirmation

T203 | PASS

apps/inventory/test_demo.py:75

ProjectDemoTests.test_failure_rolls_back_entire_seed

T204 | PASS

apps/inventory/test_demo.py:88

ProjectDemoTests.test_read_only_verifier_rejects_active_allocation_on_pending_request

4.1 Complete suite scope beyond selected references

The register lists only the methods cited by the requirement-derived scenarios. The entire permanent suite, including additional UI and regression checks, ran: 524 PASS, zero failures/errors and zero skips. No unreferenced method is automatically counted as an additional PDF requirement or a completed manual evaluation.

5. Summary, defects and outstanding verification

Scenario outcome	Count
PASS	108
FAIL	0
NOT EXECUTED	3
BLOCKED	5
Total	116

Requirement coverage	Count
PASS	72
PARTIAL	2
NOT EXECUTED	1
BLOCKED	5
Normalized requirements	80
Requirements with at least one PASS	74

No functional assertion failure was observed in the executed suite. Known limitations are evidence gaps, not silently repaired defects: physical printer/print-preview output was not exercised; the complete mobile/tablet, keyboard and screen-reader evaluation was not executed; acceptance participants, institutional policy review, signed disclosure, private instructor credential delivery, verified section/roster/contributions and live presentation evidence remain pending.

SQLite concurrency checks establish the tested one-winner outcome, but do not establish row-level locking or production-scale throughput. A different backend or deployment requires its own migration, concurrency, security and performance verification. The original working source/database were preserved; the submission process did not implement fixes.

5.1 Before the instructor demonstration

1. Fill the verified section/roster, add real contribution/leadership/peer-feedback records and complete actual AI/asset declarations and signatures under instructor policy.

2. Run user acceptance tasks and record real outcomes, issues and resolutions. Complete the phone/tablet, keyboard/screen-reader and physical-print scenarios and update only after observation.

3. Generate fresh private fictional demonstration credentials and deliver one Admin/two Basic User accounts through the instructor-designated channel.

4. Prepare the timed twelve-minute presentation plus three-minute Q&A with an identifiable actual segment for every member, record feedback and actions, and use the actual section/group ZIP naming convention.

5.2 Source reference

[R1] IT 401W - Project (Equipment Borrowing) - 1st Sem 26-27.pdf, Bulacan State University, pp.1-5. Requirement groups, business rules, logical data/workflow, technical constraints and academic evidence obligations are transcribed by meaning rather than treated as additional feature requests. The official source hash is recorded in Section 1.1. Included source code and exact T-number references establish implementation and test evidence; real screenshots are in docs/screenshots/.